

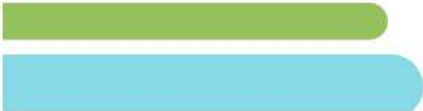
תכנות מונחה עצמים

מחלקות ועצמים





ביחידה זו נלמד:

- מהו תכנות מונחה עצמים
 - מהו עצם ומהי מחלקה
 - תכונות ופעולות של מחלקות
 - יצירת עצמים
 - הרשאות גישה
 - משתנים מסוג הפניה
 - מילת המפתח this
 - תכונות ופעולות סטטיות
 - מערך עצמים
- 
- 



ביחידה זו נלמד:

- מהו תכנות מונחה עצמים
 - מהו עצם ומהי מחלקה
 - תכונות ופעולות של מחלקות
 - יצירת עצמים
 - הרשאות גישה
 - משתנים מסוג הפניה
 - מילת המפתח `this`
 - תכונות ופעולות סטטיות
 - מערך עצמים
- 
- 

הכירו את תום

זהו תום ספיר

תום הוא תלמיד בכיתה יא 4

מספר הטלפון של תום הוא 087-6543211

במבחן הראשון תום קיבל 86

במבחן השני תום קיבל 89

במבחן השלישי תום קיבל 98

תום סיים את מטלת הביצוע



הכירו את תום

זהו תום ספיר

תום הוא תלמיד בכיתה יא 4

מספר הטלפון של תום הוא 087-6543211

במבחן הראשון תום קיבל 86

במבחן השני תום קיבל 89

במבחן השלישי תום קיבל 98

תום סיים את מטלת הביצוע

תום הוא אחד התלמידים שלי

```
string firstName = "Tom";  
string lastName = "Sapir";  
string phone = "087-6543211";  
int grade1 = 86;  
int grade2 = 89;  
int grade3 = 98;  
bool codingWork = true;
```





הכירו את כל התלמידים

```
string firstName1 = "...";  
string lastName1 = "...";  
string phone1 = "...";  
ir string firstName2 = "...";  
ir string lastName2 = "...";  
ir string phone2 = "...";  
bo in string firstName3 = "...";  
in string lastName3 = "...";  
in string phone3 = "...";  
bo in string firstName4 = "...";  
in string lastName4 = "...";  
in string phone4 = "...";  
string firstName5 = "...";  
string lastName5 = "...";  
string phone5 = "...";
```

```
string firstName6 = "...";  
string lastName6 = "...";  
string phone6 = "...";  
int grade16 = ...;  
int grade26 = ...;  
int grade36 = ...;  
bool codingWork6 = ...;
```

תכנות מונחה עצמים



- הטיפוסים הפרימיטיביים אינם מספיקים לתיאור ישויות מורכבות כמו הסטודנט שלנו, ואנו זקוקים לכמה משתנים נפרדים
- קשה לנהל תוכנית עם מספר רב כל כך של משתנים
- בנוסף, בתוכנית על סטודנטים, נרצה שתהיינה לנו פעולות הקשורות במאפייני הסטודנט – מה ממוצע ציוניו? מה הציון הגבוה ביותר? האם השתפר בציוניו?
- תכנות מונחה עצמים, מאפשר להגדיר טיפוסים חדשים, על פי הצורך
- מטיפוסים אלו נוכל ליצור עצמים (אובייקטים), כולם בעלי אותם מאפיינים (תכונות), אך ערכיהם יכולים להיות שונים זה מזה

מה מאפיין שוקולד?

יצרן

משקל (בגרם)

תוספת

מחיר

חלבי?

כשר?

רשימת המאפיינים תלויה בצורן.
במעבדת בריאות אולי יזקקו למאפיינים
כמו תאריך תפוגה, כמויות חיידיקים ועובש.
דיאטנים יזקקו אולי לכמות הקלוריות.
בבית הדפוס יזקקו לצבע עטיפה וסוג
אריזה.

מה מאפיין שוקולד?

nusela	mutela	mutela	יצרן
150	200	100	משקל (בגרם)
ללא	ללא	אגוזים	תוספת
12.30	12.30	6.40	מחיר
לא	כן	כן	חלבי?
לא	לא	כן	כשר?

עצמים מטיפוס שוקולד
לכל עצם יש את תכונות הטיפוס ולכל תכונה
יש ערך

מה מאפיין שוקולד?

nusela	mutela	mutela	יצרן
150	200	100	משקל (בגרם)
ללא	ללא	אגוזים	תוספת
12.30	12.30	6.40	מחיר
לא	כן	כן	חלבי?
לא	לא	כן	כשר?

פעולת הטיפוס – ניתן לבצע על כל עצם ממנו!

מה מאפיין שוקולד?

nusela	mutela	mutela	יצרן
150	200	100	משקל (בגרם)
ללא	ללא	אגוזים	תוספת
12.30	12.30	6.40	מחיר
לא	כן	כן	חלבי?
לא	לא	כן	כשר?

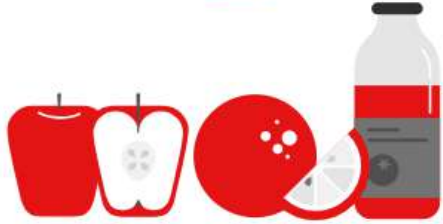
אכול(כמה גרם)

מחיר_אחרי_הנחה(כמה_הנחה)

שנה_כשרות()

פעולת הטיפוס – ניתן לבצע על כל עצם ממנו!

תכנות מונחה עצמים



○ הבסיס התפיסתי של תכנות מונחה עצמים הוא שהעולם מאורגן בעצמים

○ כל דבר הוא עצם (אובייקט)

○ לכל עצם יש מאפיינים המייצגים אותו – לעצמים מאותו טיפוס, יש אותם מאפיינים (תכונות)

○ לכל עצם יש פעולות שהוא יודע לעשות - כל העצמים מאותו טיפוס יכולים לבצע אותן פעולות



תזכורת - מחלקה - אוסף של פעולות (*ותכונות - בהמשך)

```
using System;

namespace StaticMethod
{
    class Program
    {
        static void PrintHello()
        {
            Console.WriteLine("hello");
        }

        static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}
```

תזכורת - מחלקה - אוסף של פעולות

```
using System;

namespace StaticMethod
{
    class Program
    {
        static void PrintHello()
        {
            Console.WriteLine("hello");
        }

        static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}
```

הפעולה הראשית Main, היא מיוחדת – תפקידה לנהל את זרימת התוכנית

הרצת התוכנית מתחילה תמיד ב Main והיא אשר קובעת את סדר ביצוע התוכנית


```
class Program
```

```
{
```

```
    static void Main(string[] args)
```

```
    {
```

פעולה ממחלקת Int32

```
        int n1 = int.Parse(Console.ReadLine());
```

פעולה ממחלקת Console

פעולה ממחלקת Math

```
        int n2 = Math.Abs(n1);
```

```
        Random rnd = new Random();
```

```
        int n3 = rnd.Next(5, 9);
```

פעולות ממחלקת Random

```
        int[] arr = new int[n1];
```

```
        int[,] matrix = new int[n2, n3];
```

פעולות ממחלקת Array

```
        Print(arr);
```

```
        Print(matrix);
```

פעולות במחלקה

```
        Console.WriteLine(arr.Length + matrix.GetLength(0));
```

פעולה ממחלקת Console

```
    static void Print(int[] a) {...}
```

```
    static void Print(int[,] a) {...}
```

```
}
```

```

class Program
{
    static void Main(string[] args)
    {
        int n1 = int.Parse(Console.ReadLine());
        int n2 = Math.Abs(n1);
        Random rnd = new Random();
        int n3 = rnd.Next(5, 9);
        int[] arr = new int[n1];
        int[,] matrix = new int[n2, n3];
        Print(arr);
        Print(matrix);
        Console.Write(arr.Length + matrix.GetLength(0));
    }

    static void Print(int[] a) {...}
    static void Print(int[,] a) {...}
}

```

פעולות אלו הן פעולות סטטיות –
אינן פעולות של עצמים

השתמשנו בפקודה new
ליצור עצמים מטיפוס מערך
ורנדום, ושמרנו את ההפניה
במשתנה מטיפוס המחלקה

פעולות ממחלקת Random
וממחלקת Array הופעלו על
משתנה מטיפוס המחלקה,
באמצעות סימן הנקודה

שאר הפעולות הופעלו ע"י
שם הפעולה, אם הזימון היה
לפעולה שנמצאת במחלקה
עצמה או ע"י שם המחלקה
נקודה פעולה, אם הן
במחלקה אחרת

משתנים – משתנים פרימיטיביים ומשתני הפניה

- **משתנים פרימיטיביים** (int, double, char, bool) הם משתנים שאין להם מבנה מורכב, הם מכילים ערך יחיד ונשמרים בזיכרון המיועד לקוד בו הם נמצאים, ב **stack**
- ערכיו של עצם נשמרים במקום אחר, **בזיכרון ה heap**. משתנה מטיפוס עצם אינו מכיל את העצם אלא את **ההפניה (הכתובת) של המקום בזיכרון** ה heap בו נשמרים נתוני העצם
- כאשר מתבצעת השמה למשתנה פרימיטיבי, הערך מועתק, ומושם בתוך המשתנה

משתנים – משתנים פרימיטיביים ומשתני הפניה

- **משתנים פרימיטיביים** (int, double, char, bool) הם משתנים שאין להם מבנה מורכב, הם מכילים ערך יחיד ונשמרים בזיכרון המיועד לקוד בו הם נמצאים, ב **stack**
- ערכיו של עצם נשמרים במקום אחר, **בזיכרון ה heap**. משתנה מטיפוס עצם אינו מכיל את העצם אלא את **ההפניה (הכתובת) של המקום בזיכרון** ה heap בו נשמרים נתוני העצם
- כאשר מתבצעת השמה למשתנה פרימיטיבי, הערך מועתק, ומושם בתוך המשתנה
- כאשר מתבצעת השמה למשתנה הפניה, **הערך המועתק הוא ההפנייה** (הכתובת של העצם בזיכרון), ואין העתקה או יצירה של מידע חדש

```

static void Main(string[] args)
{
    int x = 7;
    int y = 8;
    Console.WriteLine(x + " " + y);
    y = x;
    Console.WriteLine(x + " " + y);
    x += 2;
    Console.WriteLine(x + " " + y);

    int[] arr1 = { 1, 2, 3, 4, 5 };
    Print(arr1);
    int[] arr2 = { 6, 7, 8, 9 };
    Print(arr2);
    arr2 = arr1;
    Print(arr1);
    Print(arr2);
    arr1[0] = 0;
    Print(arr1);
    Print(arr2);
}

```

זיכרון ה stack

int y	8
int x	7

7 8

```

static void Main(string[] args)
{
    int x = 7;
    int y = 8;
    Console.WriteLine(x + " " + y);
    y = x;
    Console.WriteLine(x + " " + y);
    x += 2;
    Console.WriteLine(x + " " + y);

    int[] arr1 = { 1, 2, 3, 4, 5 };
    Print(arr1);
    int[] arr2 = { 6, 7, 8, 9 };
    Print(arr2);
    arr2 = arr1;
    Print(arr1);
    Print(arr2);
    arr1[0] = 0;
    Print(arr1);
    Print(arr2);
}

```

זיכרון ה stack

int y	7
int x	9

```

7 8
7 7

```



```

static void Main(string[] args)
{
    int x = 7;
    int y = 8;
    Console.WriteLine(x + " " + y);
    y = x;
    Console.WriteLine(x + " " + y);
    x += 2;
    Console.WriteLine(x + " " + y);

    int[] arr1 = { 1, 2, 3, 4, 5 };
    Print(arr1);
    int[] arr2 = { 6, 7, 8, 9 };
    Print(arr2);
    arr2 = arr1;
    Print(arr1);
    Print(arr2);
    arr1[0] = 0;
    Print(arr1);
    Print(arr2);
}

```

זיכרון ה stack

int y	7
int x	9

```

7 8
7 7
9 7

```

```

static void Main(string[] args)
{
    int x = 7;
    int y = 8;
    Console.WriteLine(x + " " + y);
    y = x;
    Console.WriteLine(x + " " + y);
    x += 2;
    Console.WriteLine(x + " " + y);

    int[] arr1 = { 1, 2, 3, 4, 5 };
    Print(arr1);
    int[] arr2 = { 6, 7, 8, 9 };
    Print(arr2);
    arr2 = arr1;
    Print(arr1);
    Print(arr2);
    arr1[0] = 0;
    Print(arr1);
    Print(arr2);
}

```

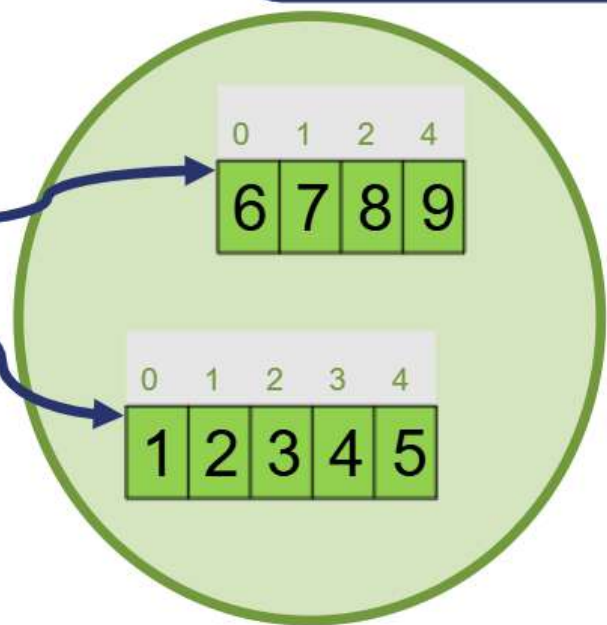
זיכרון ה stack

int[] arr2	
int[] arr1	
int y	7
int x	9

```

7 8
7 7
9 7
[1, 2, 3, 4, 5]
[6, 7, 8, 9]

```



זיכרון ה heap

```

static void Main(string[] args)
{
    int x = 7;
    int y = 8;
    Console.WriteLine(x + " " + y);
    y = x;
    Console.WriteLine(x + " " + y);
    x += 2;
    Console.WriteLine(x + " " + y);

    int[] arr1 = { 1, 2, 3, 4, 5 };
    Print(arr1);
    int[] arr2 = { 6, 7, 8, 9 };
    Print(arr2);
    arr2 = arr1;
    Print(arr1);
    Print(arr2);
    arr1[0] = 0;
    Print(arr1);
    Print(arr2);
}

```

זיכרון ה stack

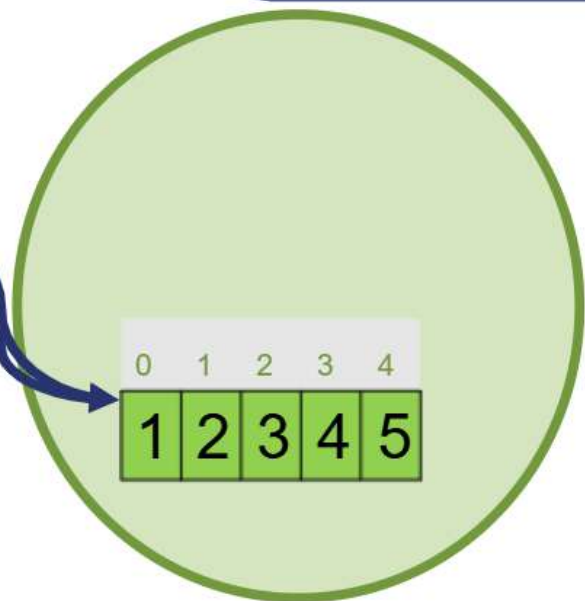
int[] arr2	
int[] arr1	
int y	7
int x	9

```

7 8
7 7
9 7
[1, 2, 3, 4, 5]
[6, 7, 8, 9]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]

```

זיכרון ה heap




```

static void Main(string[] args)
{
    int x = 7;
    int y = 8;
    Console.WriteLine(x + " " + y);
    y = x;
    Console.WriteLine(x + " " + y);
    x += 2;
    Console.WriteLine(x + " " + y);

    int[] arr1 = { 1, 2, 3, 4, 5 };
    Print(arr1);
    int[] arr2 = { 6, 7, 8, 9 };
    Print(arr2);
    arr2 = arr1;
    Print(arr1);
    Print(arr2);
    arr1[0] = 0;
    Print(arr1);
    Print(arr2);
}

```

זיכרון ה stack

int[] arr2	
int[] arr1	
int y	7
int x	9

```

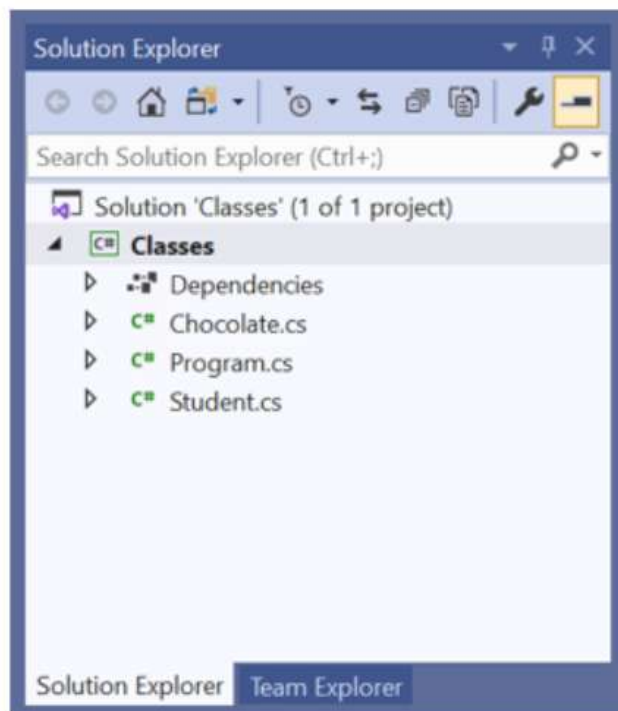
7 8
7 7
9 7
[1, 2, 3, 4, 5]
[6, 7, 8, 9]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
[0, 2, 3, 4, 5]
[0, 2, 3, 4, 5]

```

זיכרון ה heap

0	1	2	3	4
0	2	3	4	5

מחלקה – אבן הבניין של תכנות מונחה עצמים



- עד כה, כתבנו מחלקה אחת, שבה הוגדרה הפעולה Main. מחלקה זו לא הגדירה טיפוס
- השתמשנו בעצמים מטיפוס מחלקות נתונות, והשתמשנו בפעולות המחלקות כקופסא שחורה
- גם עכשיו, על מנת להריץ את התוכנית, נזקק למחלקה ראשית, בה יש את הפעולה הראשית, Main, האחראית על ביצוע האלגוריתם
- כעת, נצטרף לפרוייקט מחלקות המייצגות טיפוסים

מחלקה – אבן הבניין של תכנות מונחה עצמים

- מחלקה המייצגת טיפוס מגדירה את התכונות של עצמים מן הטיפוס ואת הפעולות שניתן להפעיל עליהם
- המחלקה הראשית מציגה פתרון לבעיה אלגוריתמית נתונה. במחלקה זו מוגדרת הפעולה הראשית Main, שמנהלת את ביצוע האלגוריתם
- הפעולה הראשית Main מופעלת באופן אוטומטי בעת הרצת המחלקה הראשית
- בתכנות מונחה עצמים מוגדרות מספר מחלקות המהוות יחד פרויקט. הפרויקט כולו מהווה פתרון לבעיה נתונה. יש להגדיר מחלקה ראשית אחת, בה מוגדרת הפעולה הראשית, פרוייקט יכול להכיל מחלקות נוספות המייצגות טיפוסים
- עצם הוא מופע של הטיפוס, יש לו ערכים לתכונות המוגדרות בטיפוס, והוא יכול לבצע את הפעולות המוגדרות בטיפוס

דוגמה המחלקה Clock

○ כדי לייצר מחלקה חדשה עלינו להוסיף קובץ חדש לפרויקט ששמו יהיה כשם המחלקה (Clock.cs) :

המילה class שמורה בשפה ומעידה שזו הגדרה של מחלקה

```
namespace Clock
{
    class Clock
    {
    }
}
```

סביבת הפרויקט

שם המחלקה

Clock

תכונות:
hour
minutes

פעולות:
ToString

פעולה המחזירה מחרוזת המתארת את העצם

דוגמה המחלקה Clock

שם המחלקה יתחיל
תמיד באות גדולה

○ כדי לייצר מחלקה חדשה עלינו להוסיף קובץ חדש לפרויקט ששמו יהיה כשם המחלקה (Clock.cs) :

המילה class שמורה בשפה ומעידה
שזו הגדרה של מחלקה

```
namespace Clock
{
    class Clock
    {
    }
}
```

סביבת הפרויקט

שם המחלקה

פה נכתוב את תכונות
ופעולות המחלקה

Clock

תכונות:

hour
minutes

פעולות:

ToString



פעולה המחזירה מחרוזת
המתארת את העצם

המחלקה Clock תכונות

```
namespace Classes
{
    class Clock
    {
        public int hours, minutes;
    }
}
```

התכונות של המחלקה Clock
הן משתנים מטיפוס int

Clock

תכונות:
hour
minutes

פעולות:
ToString

המחלקה Clock שימוש

```
namespace Classes
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            Clock c = new Clock();
```

הטיפוס

המשתנה

יצירת עצם, מופע של המחלקה

זימון פעולה בונה

Clock

תכונות:
hour
minutes

פעולות:
ToString

המחלקה Clock שימוש

```
namespace Classes
```

```
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            Clock c = new Clock();  
            c.hours = 10;  
            c.minutes = 8;  
            Console.WriteLine("The time is " + c.ToString());  
        }  
    }  
}
```

המשתנה

יצירת עצם, מופע של המחלקה

זימון פעולה בונה

הטיפוס

שינוי ערכי תכונות העצם

הפעלת פעולה על העצם, באמצעות
סימן הנקודה על המשתנה

Clock

תכונות:
hour
minutes

פעולות:
ToString

המחלקה Clock שימוש

```
namespace Classes
```

```
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            Clock c = new Clock();  
            c.hours = 10;  
            c.minutes = 8;  
            Console.WriteLine("The time is " + c.ToString());  
        }  
    }  
}
```

המשתנה

יצירת עצם, מופע של המחלקה

זימון פעולה בונה

הטיפוס

שינוי ערכי תכונות העצם

הפעלת פעולה על העצם, באמצעות סימן הנקודה על המשתנה

Clock

תכונות:
hour
minutes

פעולות:
ToString

הפעולה ToString

מחלקה ללא הגדרת ToString

```
class Clock
{
    public int hours, minutes;
}
```

```
namespace Classes
{
    class Program
    {
        static void Main(string[] args)
        {
            Clock c = new Clock();
            c.hours = 10;
            c.minutes = 8;
            Console.WriteLine("The time is " + c.ToString());
        }
    }
}
```

- הפעולות Console.Write/ Console.WriteLine מקבלות כפרמטר מחרוזת (string)
- הפעולות מבצעות המרה של הפרמטר למחרוזת, וכברירת מחדל מזמנות את הפעולה ToString
- **קיימת פעולת ToString ברירת מחדל**, הפועלת בכל מחלקה בה לא הוגדרה מחדש פעולה זו, אך היא מחזירה מחרוזת של שם הפרוייקט ושם המחלקה

הפעולה ToString

מחלקה ללא הגדרת ToString

```
class Clock
{
    public int hours, minutes;
}
```

```
namespace Classes
{
    class Program
    {
        static void Main(string[] args)
        {
            Clock c = new Clock();
            c.hours = 10;
            c.minutes = 8;
            Console.WriteLine("The time is " + c.ToString());
        }
    }
}
```

○ הפעולות Console.Write/ Console.WriteLine מקבלות כפרמטר מחרוזת (string)

○ הפעולות מבצעות המרה של הפרמטר למחרוזת, וכברירת מחדל מזמנות את הפעולה ToString

○ **קיימת פעולת ToString ברירת מחדל**, הפועלת בכל מחלקה בה לא הוגדרה מחדש פעולה זו, אך היא מחזירה מחרוזת של שם הפרוייקט ושם המחלקה

○ כדי לקבל מחרוזת כרצוננו, נוכל להגדיר מחדש את הפעולה

The time is Classes.Clock

המחלקה Clock פעולות

```
namespace Classes
{
    class Clock
    {
        public int hours, minutes;
        //פעולה משתמשת בתכונות המחלקה
        //פעולה מחזירה מחרוזת
        //המתארת את תכונות העצם
        public override string ToString()
        {
            string str = "";
            if (hours < 10)
                str = str + "0";
            str = str + hours + ":";
            if (minutes < 10)
                str = str + "0";
            str = str + minutes;

            return str;
        }
    }
}
```

המילה **override** מתחייבת, ותוסבר בהמשך

שימו לב להעדר המילה **static** מכותרת הפעולה! שימו לב לעובדה שהפעולה אינה מקבלת את העצם כפרמטר!

כזכור, פעולה על עצם מופעלת באמצעות סימן הנקודה על משתנה ההפנייה

Clock

תכונות:
hour
minutes

פעולות:
ToString


```
namespace Classes
```

```
{
```

```
    class Clock
```

```
    {
```

```
        public int hours, minutes;
```

```
        //פעולה משתמשת בתכונות המחלקה
```

```
        //פעולה מחזירה מחרוזת
```

```
        //המתארת את תכונות העצם
```

```
        public override string ToString()
```

```
        {
```

```
            string str = "";
```

```
            if (hours < 10)
```

```
                str = str + "0";
```

```
            str = str + hours + ":";
```

```
            if (minutes < 10)
```

```
                str = str + "0";
```

```
            str = str + minutes;
```

```
            return str;
```

```
        }
```

```
    }
```

```
}
```

המחלקה Clock פעולות

הפעולה מחזירה מחרוזת המתארת את העצם
במבנה hh:mm

המילה **override** מתחייבת,
ותוסבר בהמשך

שימו לב להעדר המילה
static מכותרת הפעולה!
שימו לב לעובדה שהפעולה
אינה מקבלת את העצם
כפרמטר!

כזכור, פעולה על עצם
מופעלת באמצעות סימן
הנקודה על משתנה ההפנייה

Clock

תכונות:
hour
minutes

פעולות:
ToString

המחלקה Clock שימוש

```
namespace Classes
{
    class Program
    {
        static void Main(string[] args)
        {
            Clock c = new Clock();
            c.hours = 10;
            c.minutes = 8;
            Console.WriteLine("The time is " + c.ToString());
        }
    }
}
```

המשתנה

זימון פעולה בונה

הטיפוס

שינוי ערכי תכונות העצם

הפעלת פעולה על העצם, באמצעות
סימן הנקודה על המשתנה

Clock

תכונות:
hour
minutes

פעולות:
ToString

המחלקה Clock שימוש

```
namespace Classes
{
    class Program
    {
        static void Main(string[] args)
        {
            Clock c = new Clock();
            c.hours = 10;
            c.minutes = 8;
            Console.WriteLine("The time is " + c.ToString());
        }
    }
}
```

המשתנה

זימון פעולה בונה

הטיפוס

שינוי ערכי תכונות העצם

הפעלת פעולה על העצם, באמצעות
סימן הנקודה על המשתנה

The time is 10:08

Clock

תכונות:
hour
minutes

פעולות:
ToString

שימוש בפעולה ToString בהדפסה

```
namespace Classes
{
    class Program
    {
        static void Main(string[] args)
        {
            Clock c = new Clock();

            c.hours = 10;
            c.minutes = 8;

            Console.WriteLine("The time is " + c.ToString());
        }
    }
}
```

The time is 10:08

קריאה לפעולת ToString של העצם

- כדי להדפיס את תיאור העצם, נשתמש בפעולה ToString שלו
- זו תמיד ברירת המחדל, לכן גם אם לא נציין במפורש זימון לפעולה ToString בעת הדפסת עצם תזומן **תמיד** פעולת ה ToString של העצם

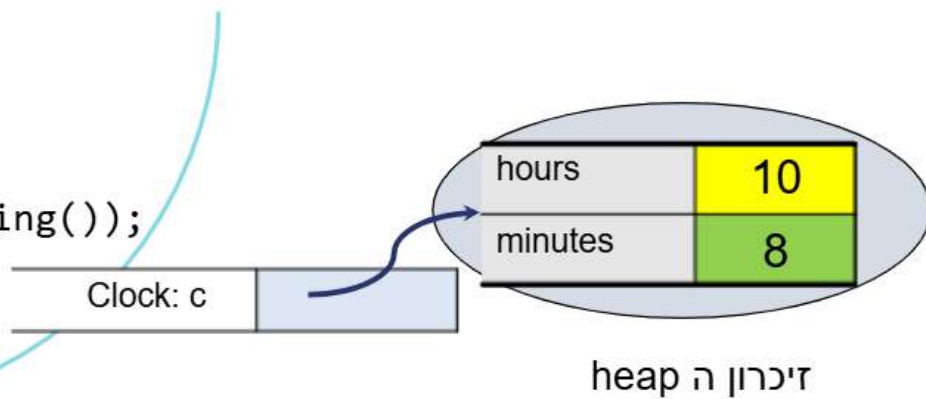
כיצד נראה הזיכרון

- בעת הגדרת המשתנה, אין לו ערך (unassigned) ולא ניתן להשתמש בו
- המשתנה נמצא בזיכרון ה stack
- העצם נוצר בזיכרון ה heap – שם מוקצה הזיכרון הנדרש לכל תכונותיו ומושם בהם ערך
- המשתנה ב stack מקבל את ההפניה (הכתובת) של העצם בזיכרון ה heap

```
namespace Clock
{
    class Program
    {
        static void Main(string[] args)
        {
            Clock c;
            c = new Clock();

            c.hours = 10;
            c.minutes = 8;

            Console.WriteLine("The time is " + c.ToString());
        }
    }
}
```



כיצד נראה הזיכרון

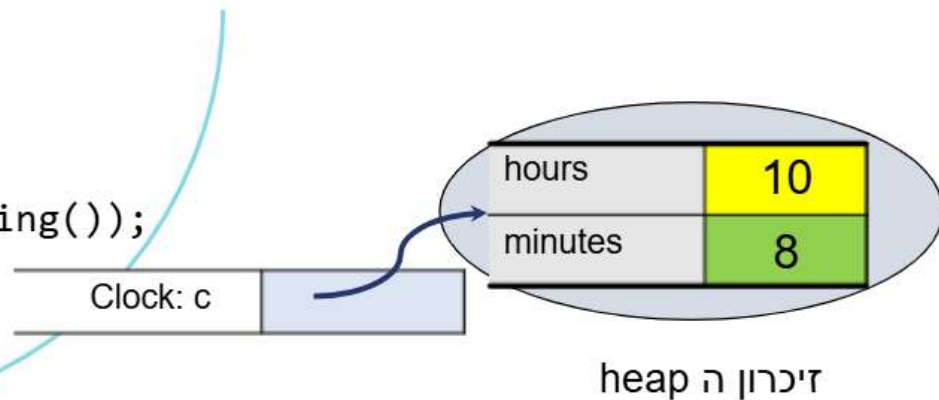
- בעת הגדרת המשתנה, אין לו ערך (unassigned) ולא ניתן להשתמש בו
- המשתנה נמצא בזיכרון ה stack
- העצם נוצר בזיכרון ה heap – שם מוקצה הזיכרון הנדרש לכל תכונותיו ומושם בהם ערך
- המשתנה ב stack מקבל את ההפניה (הכתובת) של העצם בזיכרון ה heap

```
namespace Clock
{
    class Program
    {
        static void Main(string[] args)
        {
            Clock c;
            c = new Clock();

            c.hours = 10;
            c.minutes = 8;

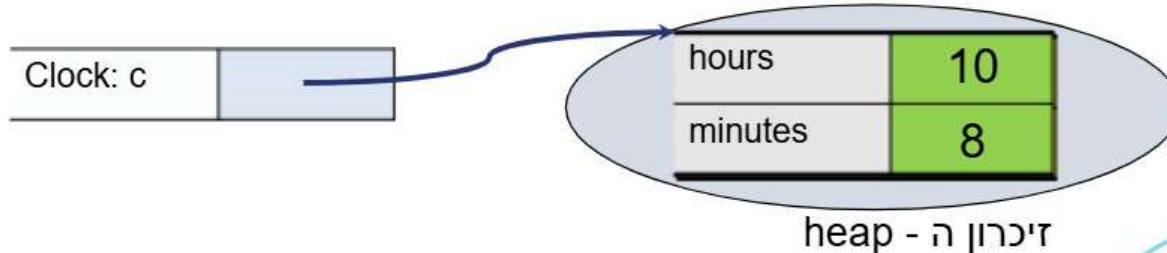
            Console.WriteLine("The time is " + c.ToString());
        }
    }
}
```

The time is 10:08



זרימת התוכנית

Clock: c



hours	10
minutes	8

זיכרון ה-heap

```
namespace clock
{
    class Program
    {
        static void Main(string[] args)
        {
            Clock c = new Clock();

            c.hours = 10;
            c.minutes = 8;

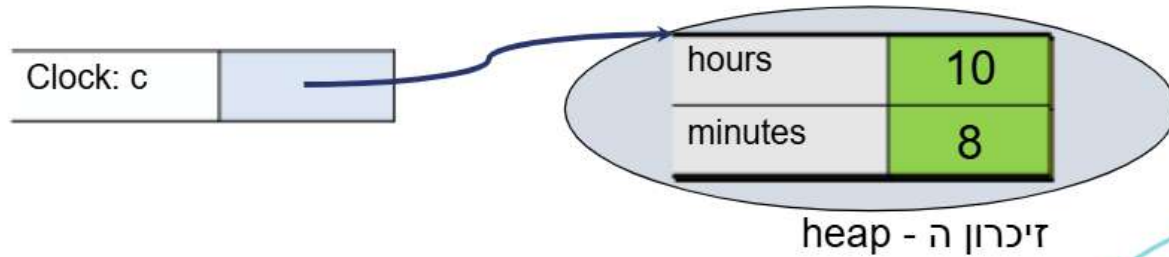
            Console.WriteLine("The time is " + c.ToString());
        }
    }
}
```

```
namespace clock
{
    class Clock
    {
        public int hours, minutes;

        public string ToString()
        {
            string str = "";
            if (hours < 10)
                str = str + "0";
            str = str + hours + ":";
            if (minutes < 10)
                str = str + "0";
            str = str + minutes;

            return str;
        }
    }
}
```

זרימת התוכנית



```
namespace clock
{
    class Program
    {
        static void Main(string[] args)
        {
            Clock c = new Clock();

            c.hours = 10;
            c.minutes = 8;

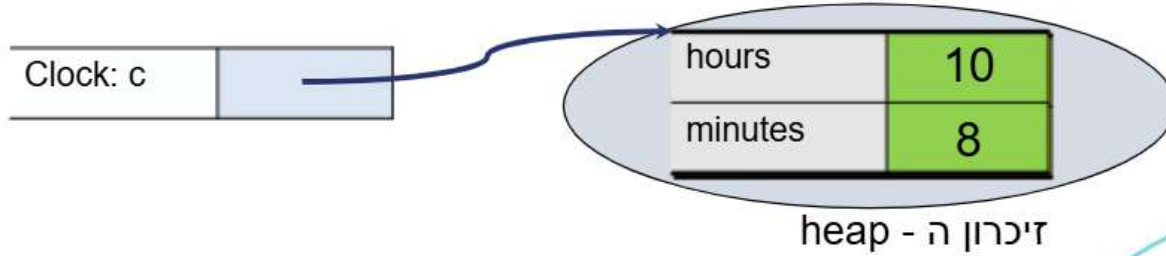
            Console.WriteLine("The time is " + c.ToString());
        }
    }
}
```

```
namespace clock
{
    class Clock
    {
        public int hours, minutes;

        public string ToString()
        {
            str = 10:08
            string str = "";
            if (hours < 10)
                str = str + "0";
            str = str + hours + ":";
            if (minutes < 10)
                str = str + "0";
            str = str + minutes;

            return str;
        }
    }
}
```


זרימת התוכנית



```
namespace clock
{
    class Program
    {
        static void Main(string[] args)
        {
            Clock c = new Clock();

            c.hours = 10;
            c.minutes = 8;

            Console.WriteLine("The time is " + c.ToString());
        }
    }
}
```

The time is 10:08

```
namespace clock
{
    class Clock
    {
        public int hours, minutes;

        public string ToString()
        {
            string str = "";
            if (hours < 10)
                str = str + "0";
            str = str + hours + ":";
            if (minutes < 10)
                str = str + "0";
            str = str + minutes;

            return str;
        }
    }
}
```

זרימת התוכנית

Clock: c

hours	10
minutes	8

זיכרון ה-heap

```
namespace clock
{
    class Program
    {
        static void Main(string[] args)
        {
            Clock c = new Clock();

            c.hours = 10;
            c.minutes = 8;

            Console.WriteLine("The time is " + c.ToString());
        }
    }
}
```

The time is 10:08

```
namespace clock
{
    class Clock
    {
        public int hours, minutes;

        public string ToString()
        {
            str = 10:08
            string str = "";
            if (hours < 10)
                str = str + "0";
            str = str + hours + ":";
            if (minutes < 10)
                str = str + "0";
            str = str + minutes;

            return str;
        }
    }
}
```

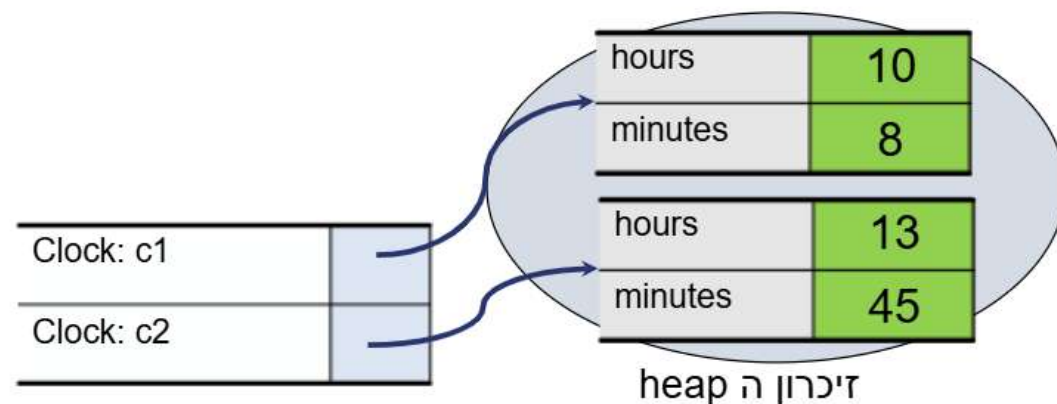
יצירת עצמים מהמחלקה Clock

```
static void Main(string[] args)
{
    Clock c1 = new Clock();
    Clock c2 = new Clock();

    c1.hours = 10;
    c1.minutes = 8;

    c2.hours = 13;
    c2.minutes = 45;

    Console.WriteLine("c1 time is: " + c1);
    Console.WriteLine("c2 time is: " + c2);
}
```



ערך משתנה ההפניה קובע לאיזה
עצם בזיכרון ניגש

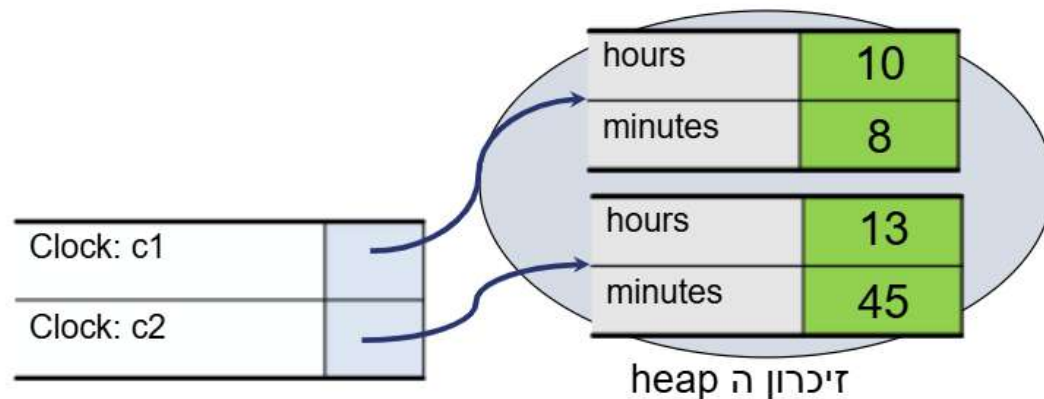
יצירת עצמים מהמחלקה Clock

```
static void Main(string[] args)
{
    Clock c1 = new Clock();
    Clock c2 = new Clock();

    c1.hours = 10;
    c1.minutes = 8;

    c2.hours = 13;
    c2.minutes = 45;

    Console.WriteLine("c1 time is: " + c1);
    Console.WriteLine("c2 time is: " + c2);
}
```



ערך משתנה ההפניה קובע לאיזה
עצם בזיכרון ניגש

```
c1 time is: 10:08
c2 time is: 13:45
```



```
static void Main(string[] args)
{
    Clock c = new Clock();

    c.hours = 30;
    c.minutes = 8;
}
```

הרשאות public ו- private

```
class Clock
{
    public int hours, minutes;

    public string ToString()
    {
        ...
    }
}
```

- לא נרצה שהתכונות שלנו יהיו חשופות ושכל אחד יוכל לשנות את ערכיהן ע"י השמה
- למשל שלא יוכלו לשים בערך של השעות מספר שאינו בין 0 ל- 23
- לכן ניתן **הרשאות** לתכונות ולפעולות של המחלקה, על פי מידת החשיפה:
- תכונה שנרצה **לחשוף לשינוי ו/או לצפייה** תישאר עם הרשאת **public**
- תכונה שנרצה שתהייה **לשינוי ו/או לצפייה רק בתוך המחלקה** תקבל הרשאת **private**

```
static void Main(string[] args)
{
    Clock c = new Clock();

    c.hours = 30;
    c.minutes = 8;
}
```

הרשאות public ו- private

```
class Clock
{
    private int hours, minutes;

    public string ToString()
    {
        ...
    }
}
```

- לא נרצה שהתכונות שלנו יהיו חשופות ושכל אחד יוכל לשנות את ערכיהן ע"י השמה
- למשל שלא יוכלו לשים בערך של השעות מספר שאינו בין 0 ל- 23
- לכן ניתן **הרשאות** לתכונות ולפעולות של המחלקה, על פי מידת החשיפה:
- תכונה שנרצה **לחשוף לשינוי ו/או לצפייה** תישאר עם הרשאת **public**
- תכונה שנרצה שתהייה **זמינה לשינוי ו/או לצפייה רק בתוך המחלקה** תקבל הרשאת **private**

הרשאות public ו- private

```
static void Main(string[] args)
{
    Clock c = new Clock();

    c.hours = 30;
    c.minutes = 8;
}
```

❌ CS0122 'Clock.hours' is inaccessible due to its protection level
❌ CS0122 'Clock.minutes' is inaccessible due to its protection level

```
class Clock
{
    private int hours, minutes;

    public string ToString()
    {
        ...
    }
}
```

- לא נרצה שהתכונות שלנו יהיו חשופות ושכל אחד יוכל לשנות את ערכיהן ע"י השמה
- למשל שלא יוכלו לשים בערך של השעות מספר שאינו בין 0 ל- 23
- לכן ניתן **הרשאות** לתכונות ולפעולות של המחלקה, על פי מידת החשיפה:
- תכונה שנרצה **לחשוף לשינוי ו/או לצפייה** תישאר עם הרשאת **public**
- תכונה שנרצה שתהייה **זמינה לשינוי ו/או לצפייה רק בתוך המחלקה** תקבל הרשאת **private**

גישה לתכונות פרטיות – פעולות אחזור וקביעה

- מקובל ורצוי שתכונות המחלקה יוגדרו כפרטיות (private)
- המחלקה מאפשרת קבלת ערך התכונה (אחזור) ושינוי ערך התכונה (קביעה) באמצעות פעולות
 - **בפעולות אחזור Getters** - שם הפעולה יהיה Get ואחריו שם התכונה:
 - לדוגמה, אם לתכונה קוראים Minutes, שם הפעולה יהיה GetMinutes
 - פעולת אחזור אינה מקבלת פרמטרים, היא מחזירה את ערך התכונה, והטיפוס המוחזר יהיה טיפוס התכונה (כאן int)
 - **בפעולות קביעה Setters** - שם הפעולה יהיה Set ואחריו שם התכונה:
 - לדוגמה, אם לתכונה קוראים Minutes, שם הפעולה יהיה SetMinutes
 - פעולת קביעה מקבלת את הערך החדש כפרמטר, וקובעת את ערך התכונה להיות ערך זה
- פעולות אחזור וקביעה, כמו שאר פעולות המחלקה המופעלות על ידי עצמים מטיפוס זה, אינן פעולות סטטיות, והמילה static לא תופיע בחתימת הפעולה


```
class Clock
```

```
{
```

```
    private int hours, minutes;
```

```
    public int GetHours()
```

```
{
```

```
        return hours;
```

```
}
```

```
    public int GetMinutes()
```

```
{
```

```
        return minutes;
```

```
}
```

```
    public void SetMinutes(int m)
```

```
{
```

```
        minutes = m;
```

```
}
```

```
    public void SetHours(int h)
```

```
{
```

```
        hours = h;
```

```
}
```

```
    public override string ToString() {...}
```

גישה לתכונות private
מימוש פעולות קובעות ומאחזרות

```
class Clock
```

```
{  
    private int hours, minutes;  
  
    public int GetHours()  
    {  
        return hours;  
    }  
  
    public int GetMinutes()  
    {  
        return minutes;  
    }  
  
    public void SetMinutes(int m)  
    {  
        minutes = m;  
    }  
  
    public void SetHours(int h)  
    {  
        hours = h;  
    }  
  
    public override string ToString() {...}  
}
```

גישה לתכונות private מימוש פעולות קובעות ומאחזרות

תפקיד פעולה מאחזרת להחזיר
ערך של תכונה בעצם

```
class Clock
```

```
{  
    private int hours, minutes;
```

```
    public int GetHours()  
    {
```

```
        return hours;  
    }
```

```
    public int GetMinutes()  
    {
```

```
        return minutes;  
    }
```

```
    public void SetMinutes(int m)  
    {
```

```
        minutes = m;  
    }
```

```
    public void SetHours(int h)  
    {
```

```
        hours = h;  
    }
```

```
    public override string ToString() {...}
```

```
}
```

גישה לתכונות private מימוש פעולות קובעות ומאחזרות

תפקיד פעולה קובעת לשים ערך
בתוך תכונה בעצם

תפקיד פעולה מאחזרת להחזיר
ערך של תכונה בעצם

```
class Clock
```

```
{
```

```
    private int hours, minutes;
```

```
    public int GetHours()
```

```
    {
```

```
        return hours;
```

```
    }
```

```
    public int GetMinutes()
```

```
    {
```

```
        return minutes;
```

```
    }
```

```
    public void SetMinutes(int m)
```

```
    {
```

```
        minutes = m;
```

```
    }
```

```
    public void SetHours(int h)
```

```
    {
```

```
        hours = h;
```

```
    }
```

```
    public override string ToString() {...}
```

```
}
```

גישה לתכונות private

מימוש פעולות קובעות ומאחזרות

תפקיד פעולה קובעת לשים ערך
בתוך תכונה בעצם

תפקיד פעולה מאחזרת להחזיר
ערך של תכונה בעצם

```
static void Main(string[] args)
```

```
{
```

```
    Clock c = new Clock();
```

```
    Console.WriteLine(c);
```

00:00

```
    c.SetHours(10);
```

```
    c.SetMinutes(8);
```

```
    Console.WriteLine(c);
```

10:08

```
    Console.WriteLine("The time is around " + c.GetHours());
```

```
}
```



```
class Clock
```

```
{
```

```
    private int hours, minutes;
```

```
    public int GetHours()
```

```
    {
```

```
        return hours;
```

```
    }
```

```
    public int GetMinutes()
```

```
    {
```

```
        return minutes;
```

```
    }
```

```
    public void SetMinutes(int m)
```

```
    {
```

```
        minutes = m;
```

```
    }
```

```
    public void SetHours(int h)
```

```
    {
```

```
        hours = h;
```

```
    }
```

```
    public override string ToString() {...}
```

```
}
```

גישה לתכונות private

מימוש פעולות קובעות ומאחזרות

תפקיד פעולה קובעת לשים ערך
בתוך תכונה בעצם

תפקיד פעולה מאחזרת להחזיר
ערך של תכונה בעצם

```
static void Main(string[] args)
```

```
{
```

```
    Clock c = new Clock();
```

```
    Console.WriteLine(c);
```

00:00

```
    c.SetHours(10);
```

```
    c.SetMinutes(8);
```

```
    Console.WriteLine(c);
```

10:08

```
    Console.WriteLine("The time is around " + c.GetHours());
```

```
}
```

The time is around 10

```
class Clock
```

```
{
```

```
    private int hours, minutes;
```

```
    public int GetHours()
```

```
    {
```

```
        return hours;
```

```
    }
```

```
    public int GetMinutes()
```

```
    {
```

```
        return minutes;
```

```
    }
```

```
    public void SetMinutes(int m)
```

```
    {
```

```
        minutes = m;
```

```
    }
```

```
    public void SetHours(int h)
```

```
    {
```

```
        hours = h;
```

```
    }
```

```
    public override string ToString() {...}
```

```
}
```

גישה לתכונות private

מימוש פעולות קובעות ומאחזרות

תפקיד פעולה קובעת לשים ערך
בתוך תכונה בעצם

תפקיד פעולה מאחזרת להחזיר
ערך של תכונה בעצם

```
static void Main(string[] args)
```

```
{
```

```
    Clock c = new Clock();
```

```
    Console.WriteLine(c);
```

00:00

```
    c.SetHours(10);
```

```
    c.SetMinutes(8);
```

```
    Console.WriteLine(c);
```

10:08

```
    Console.WriteLine("The time is around " + c.GetHours());
```

```
}
```

The time is around 10

```
class Clock
```

```
{
```

```
    private int hours, minutes;
```

```
    public int GetHours()
```

```
    {
```

```
        return hours;
```

```
    }
```

```
    public int GetMinutes()
```

```
    {
```

```
        return minutes;
```

```
    }
```

```
    public void SetMinutes(int m)
```

```
    {
```

```
        minutes = m;
```

```
    }
```

```
    public void SetHours(int h)
```

```
    {
```

```
        hours = h;
```

```
    }
```

```
    public override string ToString() {...}
```

```
}
```

גישה לתכונות private

מימוש פעולות קובעות ומאחזרות

תפקיד פעולה קובעת לשים ערך
בתוך תכונה בעצם

תפקיד פעולה מאחזרת להחזיר
ערך של תכונה בעצם

```
static void Main(string[] args)
```

```
{
```

```
    Clock c = new Clock();
```

```
    Console.WriteLine(c);
```

00:00

```
    c.SetHours(10);
```

```
    c.SetMinutes(8);
```

עדיין אפשר לשלוח ערך לא תקין!

```
    Console.WriteLine(c);
```

10:08

```
    Console.WriteLine("The time is around " + c.GetHours());
```

The time is around 10



פעולות קובעות

- כיצד נתמודד עם ערכים שאינם תקינים?
- קיימות כמה אפשרויות, ובין היתר:

- אם הערך לא תקין, הפעולה לא תשנה את ערך התכונה
- אם הערך לא תקין, הפעולה תקבע ערך ברירת מחדל
- הפעולה יכולה להחזיר ערך בוליאני – אמת אם בוצעה ההשמה למשתנה התכונה ושקר אם לא
- ניתן להדפיס הודעת שגיאה

שימו לב כי כאשר נכתב בשאלות/בבחינות כי נתונות לכם פעולות Set, הכוונה בפעולות קובעות ללא בדיקת ערכים ושאינן מחזירות ערך, אלא אם נכתב מפורשות אחרת



פעולות קובעות

- כיצד נתמודד עם ערכים שאינם תקינים?
- קיימות כמה אפשרויות, ובין היתר:
- אם הערך לא תקין, הפעולה לא תשנה את ערך התכונה
- אם הערך לא תקין, הפעולה תקבע ערך ברירת מחדל
- הפעולה יכולה להחזיר ערך בוליאני – אמת אם בוצעה ההשמה למשתנה התכונה ושקר אם לא
- ניתן להדפיס הודעת שגיאה

שימו לב כי כאשר נכתב בשאלות/בבחינות כי נתונות לכם פעולות Set, הכוונה בפעולות קובעות ללא בדיקת ערכים ושאינן מחזירות ערך, אלא אם נכתב מפורשות אחרת

פעולות קובעות מימושים מתוקנים

```
class Clock
{
    private int hours, minutes;

    public int GetHours() { ... }
    public int GetMinutes() { ... }

    public void SetMinutes(int m)
    {
        if (m >= 0 && m <= 59)
            minutes = m;
        else
            Console.WriteLine("Minutes have to be between 0-59");
    }

    public void SetHours(int h)
    {
        if (h >= 0 && h <= 23)
            hours = h;
        else
            Console.WriteLine("Hours have to be between 0-23");
    }

    public string ToString() { ... }
}
```

ל Main אין אינדיקציה לכך
שהערך אינו תקין!

```
static void Main(string[] args)
{
    Clock c = new Clock();

    c.SetHours(30);
    c.SetMinutes(8);

    Console.WriteLine(c.ToString());
}
```

Hours have to be between 0-23
00:08

המחלקה אחראית לתקינות הנתונים

```

static void Main(string[] args)
{
    Clock c = new Clock();
    bool res = false;
    int hours, minutes;

    while (res == false)
    {
        Console.Write("Enter hours: ");
        hours = int.Parse(Console.ReadLine());
        res = c.SetHours(hours);
    }

    res = false;
    while (res == false)
    {
        Console.Write("Enter minutes: ");
        minutes = int.Parse(Console.ReadLine());
        res = c.SetMinutes(minutes);
    }

    Console.WriteLine(c.ToString());
}

```

וכיצד ה-Main ידע האם לקלוט נתונים מחדש?

הפעולה הקובעת תחזיר bool
כאינדיקציה להצלחה או כישלון

```

Enter hours: 30
Hours have to be between 0-23
Enter hours: 8
Enter minutes: 80
Minutes have to be between 0-59
Enter minutes: 5
08:05

```

פעולות קובעות יחזירו bool

```
public bool SetMinutes(int m)
{
    if (m >= 0 && m <= 59)
    {
        minutes = m;
        return true;
    }
    else
    {
        Console.WriteLine("Minutes have to be between 0-59");
        return false;
    }
}
```


האם נרצה פעולות קובעות ומאחזרות עבור כל תכונה?



○ פעולות קובעות – רק לתכונות שנרצה לאפשר בהן שינוי

למשל, לא נרצה לתת אפשרות לשינוי מספר זהות

○ פעולות מאחזרות – רק לתכונות שנרצה לחשוף אותן למשתמשים

למשל, אם לא נרצה לחשוף את תאריך הלידה המדויק של המשתמש, אלא רק את גילו

נכתוב פעולות קובעות ומאחזרות לפי הצורך, ולא באופן אוטומטי

פעולות קובעות ומאחזרות סיכום

```
public bool SetMinutes(int m)
{
    if (m >= 0 && m <= 59)
    {
        minutes = m;
        return true;
    }
    else
    {
        Console.WriteLine("...");
        return false;
    }
}

public int GetHours()
{
    return hours;
}
```

○ כדי לאפשר השמת ערך בתכונה שהיא **private** נכתוב **פעולה קובעת**:

1. תהיה **public**
2. תקבל כפרמטר את הערך המבוקש
3. תבצע את בדיקות התקינות על הפרמטר
4. תעדכן את הערך המתאים בתכונה
5. תחזיר **bool**

○ כדי לאפשר קבלת ערך תכונה שהיא **private** נכתוב **פעולה מאחזרת**:

1. תהיה **public**
2. תחזיר את התכונה

האם תמיד נגדיר את התכונות כ **private**?



האם תמיד נגדיר את התכונות כ `private`?

- האם אנו יודעים מהן תכונות מחלקת Random או Array?
 - האם הדבר מנע מאיתנו להשתמש בפעולות המחלקה?
 - מדוע לתכונה Length של המערך יש גישה ישירה?
-
- רשימת פעולות המחלקה (ממשק המחלקה) כוללת את חתימת הפעולות, טענות הכניסה והיציאה, ואת הערך המוחזר
 - ניתן להשתמש במחלקה ללא ידיעה על תכונות המחלקה
-
- התכונה Length של המערך איננה ניתנת לשינוי, ולכן הגישה הישירה מותרת!

האם תמיד נגדיר את התכונות כ `private`?

- ישנם מקרים בהם בשלב כתיבת המחלקה אין צורך בבדיקות תקינות עבור ערכי התכונות
 - לדוגמה, יתכן כי גיל האדם אינו רלבנטי לקניית מוצר מסויים, אך בשלב מאוחר יותר נחקק חוק שאין למכור את המוצר לקטינים
 - נרצה להוסיף בדיקות תקינות (מגיל 18 ומעלה)
- לעיתים, יהיה צורך לשנות קוד מסוים במחלקה, כמו למשל ייצוג הנתונים במערך במקום נתונים בודדים
 - מניעת גישה ישירה לנתונים עצמם ומתן גישה באמצעות פעולות `Get`, מאפשרת תיקון במחלקה שלא תשפיע על המשתמשים במחלקה זו

```
class Clock
```

```
{  
    private int hours, minutes;
```

```
    public int GetHours()  
    {
```

```
        return hours;  
    }
```

```
    public int GetMinutes()  
    {
```

```
        return minutes;  
    }
```

```
    public void SetMinutes(int m)  
    {
```

```
        minutes = m;  
    }
```

```
    public void SetHours(int h)  
    {
```

```
        hours = h;  
    }
```

```
    public override string ToString() {...}
```

```
}
```

למה התכונות תמיד יהיו private?

מימוש שונה למחלקה Clock

```
static void Main(string[] args)
```

```
{  
    Clock c = new Clock();
```

```
    c.SetHours(30);  
    c.SetMinutes(8);
```

```
    Console.WriteLine(c);  
}
```

ל Main אין אינדיקציה למבנה
האמיתי של המחלקה

Hours have to be between 0-23
00:08

שימוש במחלקה אינו מחייב לדעת את פרטי המימוש

למה התכונות תמיד יהיו private?

מימוש שונה למחלקה Clock

ל Main אין אינדיקציה למבנה
האמיתי של המחלקה

```
static void Main(string[] args)
{
    Clock c = new Clock();

    c.SetHours(30);
    c.SetMinutes(8);

    Console.WriteLine(c);
}
```

Hours have to be between 0-23
00:08

שימוש במחלקה אינו מחייב לדעת את פרטי המימוש

מספיק לדעת את שם הפעולה, מהם הנתונים שיש להעביר ומה היא עושה

```
class Clock
{
    private int[] arr= new int[2];

    public int GetHours()
    {
        return arr[0]; //hours
    }

    public int GetMinutes()
    {
        return arr[1]; //minutes
    }

    public void SetMinutes(int m)
    {
        arr[1]= m; //minutes
    }

    public void SetHours(int h)
    {
        arr[0]= h; //hours
    }

    public override string ToString() {...}
}
```

תכונות- תמיד private!

○ גם אם בעת כתיבת המחלקה אין הגבלות תקינות על תכונה, **הרשאת הגישה תהיה תמיד private** ונספק פעולות קובעות ומאחזרות

שתי סיבות לאופן כתיבה זה:

1. שאם **בעתיד תתווסף** בדיקת תקינות, מי שמשתמש במחלקה לא יצטרך לשנות את הקוד שלו, כיוון שממילא השתמש בפעולות הקובעות, וכל שינוי בגוף הפעולה יהיה שקוף מבחינתו
2. **אחידות** - הפניה לכל התכונות במחלקה תהייה באופן זהה, באמצעות פעולות קובעות ומאחזרות

פרמטרים בעלי שם משמעותי

- פעולות קובעות מקבלות פרמטרים עבור ערכי התכונות
- קיצרנו את שמות הפרמטרים הללו, כדי לא לייצר התנגשות בין שם התכונה לשם הפרמטר
- נרצה לקרוא לפרמטרים בשמות של התכונות עצמן

```
class Clock
{
    private int hours;
    private int minutes;

    public void SetMinutes (int minutes)
    {
        minutes = minutes;
    }
    public void SetHours (int hours)
    {
        hours = hours;
    }
}
```

- הבעיה :
- הקומפילר לא יודע מתי אנחנו מתייחסים לפרמטר hours ומתי לתכונה hours
- אם נכתוב כך, ערך הפרמטר יתעדכן ולא ערך התכונה!

פרמטרים בעלי שם משמעותי

```
static void Main(string[] args)
{
    Clock c = new Clock();
    c.SetMinutes(30);
}
```

```
class Clock
{
    ...
    public void SetMinutes(int m)
    {
        minutes = m;
    }
    ...
}
```

העצם אינו מתקבל כפרמטר, ואין לנו משתנה הפניה אליו!

כיצד נתייחס לעצם בפעולה במחלקה?

```
static int first(int[] arr)
{
    return arr[0];
}
```

כאשר מקבלים הפניה לעצם כפרמטר, ניתן להתייחס אליו בתוך הפעולה, תוך שימוש בשם המשתנה

פרמטרים בעלי שם משמעותי

```
static void Main(string[] args)
{
    Clock c = new Clock();
    c.SetMinutes(30);
}
```

בהפעלת הפעולה, אנו קובעים את העצם עליו תתבצע, באמצעות סימן הנקודה על ההפניה

```
class Clock
{
    ...
    public void SetMinutes(int m)
    {
        minutes = m;
    }
    ...
}
```

העצם אינו מתקבל כפרמטר, ואין לנו משתנה הפניה אליו!

כיצד נתייחס לעצם בפעולה במחלקה?

```
static int first(int[] arr)
{
    return arr[0];
}
```

כאשר מקבלים הפניה לעצם כפרמטר, ניתן להתייחס אליו בתוך הפעולה, תוך שימוש בשם המשתנה

פרמטרים בעלי שם משמעותי

```
static void Main(string[] args)
{
    Clock c = new Clock();
    c.SetMinutes(30);
}
```

```
class Clock
{
    ...
    public void SetMinutes(int m)
    {
        minutes = m;
    }
    ...
}
```

```
static int first(int[] arr)
{
    return arr[0];
}
```

היכן נשמרת במחלקה ההפניה לעצם בזיכרון?


```
class Clock
{
```

```
    private int hours;
    private int minutes;
```

```
    public void SetMinutes(int m)
    {
        minutes = m;
    }
```

this היא
מילה שמורה,
וכאן יש בה
את ההפניה
לעצם הנוכחי

```
    public void SetMinutes(int minutes )
    {
        this.minutes = minutes;
    }
```

ונכתוב אותה כך:

שימוש בפרמטר

השימוש ב- this מאפשר העברת פרמטרים בשם התכונה עצמה

המילה השמורה this

נשנה את הפעולה SetMinutes

```
class Clock
{
    private int hours, minutes;

    public void SetHours(int hours)
    {
        this.hours = hours;
    }
    public void SetMinutes(int minutes)
    {
        this.minutes = minutes;
    }
    public override string ToString()
    {
        string str = "";
        if (this.hours < 10)
            str = str + "0";
        str = str + this.hours + ":";
        if (this.minutes < 10)
            str = str + "0";
        str = str + this.minutes;

        return str;
    }
}
```

המילה השמורה `this`

```
static void Main(string[] args)
{
    Clock c1 = new Clock();
    Clock c2 = new Clock();

    c1.SetHourse(5);

    Console.WriteLine(c2.ToString());
}
```

המילה השמורה `this`

```
class Clock
{
    private int hours, minutes;

    public void SetHours(int hours)
    {
        this.hours = hours;
    }
    public void SetMinutes(int minutes)
    {
        this.minutes = minutes;
    }
    public override string ToString()
    {
        string str = "";
        if (this.hours < 10)
            str = str + "0";
        str = str + this.hours + ":";
        if (this.minutes < 10)
            str = str + "0";
        str = str + this.minutes;

        return str;
    }
}
```

```
static void Main(string[] args)
{
    Clock c1 = new Clock();
    Clock c2 = new Clock();

    c1.SetHourse(5);

    Console.WriteLine(c2.ToString());
}
```

המילה השמורה this

```
class Clock
{
    private int hours, minutes;

    public void SetHours(int hours)
    {
        this.hours = hours;
    }
    public void SetMinutes(int minutes)
    {
        this.minutes = minutes;
    }
    public override string ToString()
    {
        string str = "";
        if (this.hours < 10)
            str = str + "0";
        str = str + this.hours + ":";
        if (this.minutes < 10)
            str = str + "0";
        str = str + this.minutes;

        return str;
    }
}
```

```
static void Main(string[] args)
{
    Clock c1 = new Clock();
    Clock c2 = new Clock();

    c1.SetHourse(5);

    Console.WriteLine(c2.ToString());
}
```

המילה השמורה this

```
class Clock
{
    private int hours, minutes;

    public void SetHours(int hours)
    {
        this.hours = hours;
    }
    public void SetMinutes(int minutes)
    {
        this.minutes = minutes;
    }
    public override string ToString()
    {
        string str = "";
        if (this.hours < 10)
            str = str + "0";
        str = str + this.hours + ":";
        if (this.minutes < 10)
            str = str + "0";
        str = str + this.minutes;

        return str;
    }
}
```

```
static void Main(string[] args)
{
    Clock c1 = new Clock();
    Clock c2 = new Clock();

    c1.SetHourse(5);

    Console.WriteLine(c2.ToString());
}
```

this מתייחס לעצם שעליו הפעילו את הפעולה

הפעולה הבונה – **בנאי** Constructor

מחלקה היא אב טיפוס, תבנית ליצירת עצמים
יצירת עצם מתבצעת

תוך שימוש במילה השמורה **new**
וזימון הפעולה הבונה:

```
Clock c1 = new Clock();
```

אבל לא כתבנו במחלקה פעולה בונה!



פעולה בונה ברירת מחדל

- עד כה, בנינו עצמים מטיפוס Clock למרות שלא כתבנו פעולה בונה
- בכל מחלקה, בה לא כתבנו פעולה בונה, קיימת פעולה בונה ברירת מחדל
 - פעולה זו אינה מקבלת פרמטרים
 - הפעולה שמה בתכונות העצם ערכי ברירת מחדל:

```
class Clock
{
    private int hours, minutes;

    public int  GetHours() {...}
    public int  GetMinutes() {...}
    public bool SetMinutes(int minutes) {...}
    public bool SetHours(int hours) {...}

    public override string ToString() {...}
}
```

- int – 0
- double – false
- bool – false
- char – תו שערך 0
- null - הפניה

פעולה בונה ברירת מחדל דוגמה

הצהרה על עצם מטיפוס Clock, שיקבל את ההפניה לעצם החדש שיווצר

```
class Clock
{
    private int hours, minutes;

    public int GetHours() {...}
    public int GetMinutes() {...}
    public bool SetMinutes(int minutes) {...}
    public bool SetHours(int hours) {...}

    public override string ToString() {...}
}
```

```
static void Main(string[] args)
{
    Clock c = new Clock();
    Console.WriteLine(c);
}
```

שימוש בפעולה
בונה ברירת מחדל

Clock: c

hours	0
minutes	0

זיכרון ה heap

הפעולה הבונה – בנאי

Constructor

תפקיד הפעולה הבונה, לבנות את העצם

המשמעות – עליה למצוא שטחי זיכרון ובהם
לשמור את ערכי התכונות

שם הפעולה הבונה הוא שם המחלקה ואין לה
ערך מוחזר (וגם לא void):

```
public Clock(....) {....}
```



פעולה בונה ללא פרמטרים דוגמה

```
class Clock
{
    private int hours, minutes;

    public Clock()
    {
        Console.WriteLine("Creating a new clock");
        this.hours = 0;
        this.minutes = 0;
    }

    public int GetHours() {...}
    public int GetMinutes() {...}
    public bool SetMinutes(int m) {...}
    public bool SetHours(int h) {...}

    public override string ToString() {...} }
```

ניתן להגדיר פעולה בונה ללא פרמטרים, זו אינה הפעולה ברירת המחדל. נוכל לאתחל את תכונות בערכים כרצוננו ולהוסיף הוראות נוספות, לפי הצורך

שם הפעולה הבונה כשם המחלקה

לפעולה בונה אין ערך מוחזר (למרות ההשמה למשתנה כאשר מזמנים אותה)

פעולה בונה ללא פרמטרים דוגמה

```
class Clock
{
    private int hours, minutes;

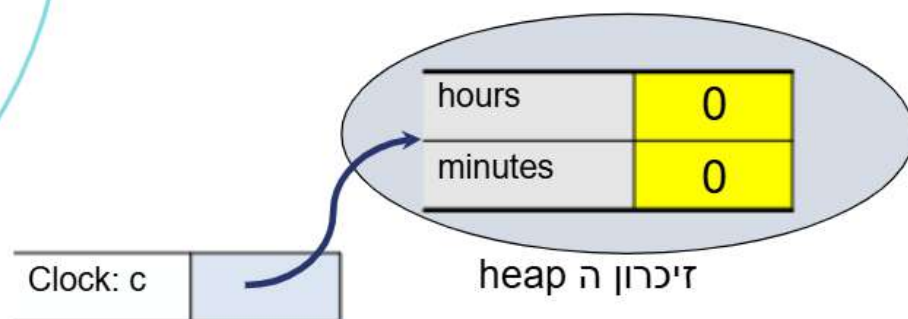
    public Clock()
    {
        Console.WriteLine("Creating a new clock");
        this.hours = 0;
        this.minutes = 0;
    }

    public int GetHours() {...}
    public int GetMinutes() {...}
    public bool SetMinutes(int m) {...}
    public bool SetHours(int h) {...}

    public override string ToString() {...} }

```

```
static void Main(string[] args)
{
    Clock c = new Clock();
    Console.WriteLine(c);
}
```



פעולה בונה ללא פרמטרים דוגמה

```
class Clock
{
    private int hours, minutes;

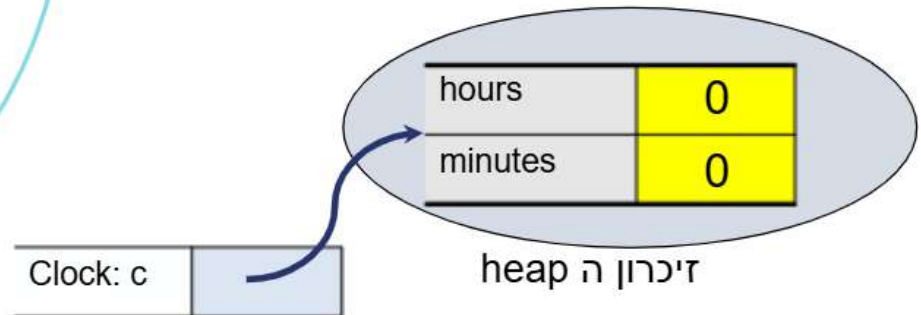
    public Clock()
    {
        Console.WriteLine("Creating a new clock");
        this.hours = 0;
        this.minutes = 0;
    }

    public int GetHours() {...}
    public int GetMinutes() {...}
    public bool SetMinutes(int m) {...}
    public bool SetHours(int h) {...}

    public override string ToString() {...} }
}
```

```
static void Main(string[] args)
{
    Clock c = new Clock();
    Console.WriteLine(c);
}
```

Creating a new clock
00:00



פעולה בונה המקבלת ערכים לתכונות

```
public Clock()  
{  
    hours = 10;  
    minutes = 30;  
}
```



```
public Clock(int hours, int minutes)  
{  
    this.hours = hours;  
    this.minutes = minutes;  
}
```

- היינו רוצים לייצר עצם כך שיהיו לו ערכים עם יצירתו, לפי הצורך (ולא ערכי ברירת מחדל)
- ניתן לכתוב פעולה בונה עם פרמטרים
- נשלח בפרמטרים ערכים התחלתיים לפעולה הבונה ונאתחל בהם את תכונות העצם בתוך הפעולה הבונה



```
class Clock
```

```
{
```

```
    private int minutes, hours;
```

```
    public Clock() {
```

```
        Console.WriteLine("Creating a Clock");
```

```
        this.hours = 0;
```

```
        this.minutes = 0;
```

```
    }
```

```
    public Clock(int hours, int minutes) {
```

```
        Console.WriteLine("Creating a Clock with parameters");
```

```
        this.hours = hours;
```

```
        this.minutes = minutes;
```

```
    }
```

```
    public override string ToString() {...}
```

```
}
```

```
static void Main(string[] args)
```

```
{
```

```
    Clock c1 = new Clock();
```

```
    Clock c2 = new Clock(12, 30);
```

```
    Console.WriteLine("c1 is " + c1);
```

```
    Console.WriteLine("c2 is " + c2);
```

```
}
```

פעולה בונה המקבלת ערכים לתכונות

דוגמה

Creating a clock

Creating a clock with parameters

c1 is 00:00

c2 is 12:30



זיכרון ה heap

Clock: c1	
Clock: c2	

פעולה בונה המקבלת ערכים לתכונות

דוגמה

האם חובה לממש פעולה בונה זו? הרי ראינו בתחילת הפרק שהקוד עובד גם בלעדיה!

```
class Clock
{
    private int minutes, hours;
    public Clock() {
        Console.WriteLine("Creating a Clock");
        this.hours = 0;
        this.minutes = 0;
    }

    public Clock(int hours, int minutes) {
        Console.WriteLine("Creating a Clock with parameters");
        this.hours = hours;
        this.minutes = minutes;
    }
    public override string ToString() {...}
}
```

```
static void Main(string[] args)
{
    Clock c1 = new Clock();
    Clock c2 = new Clock(12, 30);
    Console.WriteLine("c1 is " + c1);
    Console.WriteLine("c2 is " + c2);
}
```

כבר אין מתנות חינם!

כאן, אפשר לוותר על הפעולה הבונה חסרת הפרמטרים ולזמן את הפעולה הבונה עם הפרמטרים עם הערכים 0,0

אם רוב השעונים נבנים עם הערכים 0,0, אין מקום להכביד על המשתמשים בכתיבתם. במקרים אלו ואחרים, נוסיף למחלקה גם פעולה בונה ללא פרמטרים

```
class Clock
{
    private int minutes, hours;

    public Clock() {...}
    public Clock(int hours, int minute) {...}

    public override string ToString() {...}
}
```

```
static void Main(string[] args)
{
    Clock c1 = new Clock();
    Clock c2 = new Clock(12, 30);
}
```

ברגע שכותבים ולו פעולה בונה אחת בעצמנו, הפעולה הבונה ברירת מחדל, שקיבלנו במתנה, נלקחת מאיתנו

הפעולה הבונה - הבנאי

```
class Clock
{
    private int minutes, hours;

    public Clock() {...}
    public Clock(int hours, int minutes) {...}

    public override string ToString() {...}
}
```

```
static void Main(string[] args)
{
    Clock c1 = new Clock();
    Clock c2 = new Clock(12, 30);
}
```

- במחלקה שתי פעולות בונות
- המהדר יודע לאיזו פעולה לפנות על פי חתימת הפעולות – רשימת הפרמטרים
- העמסת פעולות **overloading**:
פעולות בעלות אותו שם וחתימה שונה – רשימת הפרמטרים חייבת להיות שונה בין פעולה לפעולה – אם במספר הפרמטרים או בטיפוס הפרמטרים או בסדר הפרמטרים

```
class Clock
```

```
{
```

```
    private int minutes, hours;
```

```
    public Clock() {...}
```

```
    public Clock(int hours, int minutes)
```

```
    {
```

```
        Console.WriteLine("Inside1 Clock(hours,minutes)");
```

```
        Console.WriteLine("this1 = " + this);
```

```
        this.hours = hours;
```

```
        this.minutes = minutes;
```

```
        Console.WriteLine("this2 = " + this);
```

```
        Console.WriteLine("Inside1 Clock(hours,minutes)");
```

```
    }
```

```
    public override string ToString() {...}
```

```
}
```

הוספנו לבנאי פעולות הדפסה – מה ערכו של this בתחילת הפעולה? מה ערכי התכונות לפני ההשמה?

הדפסת this היא בדיוק כמו הדפסת משתנה הפניה לעצם

```
Clock c = new Clock(13, 45);
```

```
Inside1 Clock(hours,minutes)
```

```
this1 = 00:00
```

```
this2 = 13:45
```

```
Inside1 Clock(hours,minutes)
```



```
class Clock
```

```
{
```

```
    private int minutes, hours;
```

```
    public Clock() {...}
```

```
    public Clock(int hours, int minutes)
```

```
    {
```

```
        Console.WriteLine("Inside1 Clock(hours,minutes)");
```

```
        Console.WriteLine("this1 = " + this);
```

```
        this.hours = hours;
```

```
        this.minutes = minutes;
```

```
        Console.WriteLine("this2 = " + this);
```

```
        Console.WriteLine("Inside1 Clock(hours,minutes)");
```

```
    }
```

```
    public override string ToString() {...}
```

```
}
```

הוספנו לבנאי פעולות הדפסה – מה ערכו של this בתחילת הפעולה? מה ערכי התכונות לפני ההשמה?

הדפסת this היא בדיוק כמו הדפסת משתנה הפניה לעצם

בשלב ראשון מוקצים שטחי זיכרון עם ערכי ברירת מחדל, רק אח"כ מבוצע קוד הבנאי

```
Clock c = new Clock(13, 45);
```

```
Inside1 Clock(hours,minutes)
```

```
this1 = 00:00
```

```
this2 = 13:45
```

```
Inside1 Clock(hours,minutes)
```

פעולה הבונה – סיכום ביניים

- בכל מחלקה המייצרת עצמים, תהיה לפחות פעולה בונה אחת
- אם לא הוגדרה אף פעולה בונה, ניתן להשתמש בפעולה בונה ברירת מחדל שאינה מקבלת פרמטרים
- שם הפעולה הבונה הוא שם המחלקה, ואין בחתימה שלה ערך מוחזר
- הפעולה הבונה מופעלת על ידי `new`
- הפעולה הבונה:
 - גורמת להקצאת שטחי זיכרון מתאימים לתכונות העצם
 - שומרת את המקום בזיכרון (ההפניה) במשתנה `this`
 - מאתחלת בשלב ראשון את הערכים לערכי ברירת המחדל (`null`, `0`)
 - מבצעת את הוראות הפעולה הבונה
- הפעולה הבונה מחזירה את ההפניה למקום בו העצם נמצא בזיכרון

העברה בין פעולות בונות

○ קיימת כפילות קוד בפעולות הבונות

○ מילת המפתח `this`, יכולה לשמש גם לזימון של בנאי אחר במחלקה

```
class Clock
{
    private int minutes, hours;

    public Clock()
    {
        this.hours = 0;
        this.minutes = 0;
    }

    public Clock(int hours, int minutes)
    {
        this.hours = hours;
        this.minutes = minutes;
    }

    public override string ToString()
    {...}
}
```

העברה בין פעולות בונות

○ קיימת כפילות קוד בפעולות הבונות

○ מילת המפתח `this`, יכולה לשמש גם לזימון של בנאי אחר במחלקה

○ הבנאי המזומן עם מילת המפתח `this` נכתב לאחר נקודותיים ולפני גוף הפעולה הבונה, ומבוצע לפנייה

```
class Clock
{
    private int minutes, hours;

    public Clock()
    {
        this.hours = 0;
        this.minutes = 0;
    }

    public Clock(int hours, int minutes)
    {
        this.hours = hours;
        this.minutes = minutes;
    }

    public override string ToString()
    {...}
}
```


העברה בין פעולות בונות

```
class Clock
{
    private int minutes, hours;

    public Clock(): this(0,0)
    {
    }

    public Clock(int hours, int minutes)
    {
        this.hours = hours;
        this.minutes = minutes;
    }

    public override string ToString()
    {...}
}
```

The diagram illustrates the flow of the 'this' reference. A green box highlights the 'this(0,0)' call in the parameterless constructor. Two green arrows originate from this box: one points to the 'this' keyword in the parameterized constructor's body, and the other points to the 'this.hours' and 'this.minutes' assignments in the same constructor's body. Another green box highlights these assignments. This visualizes how the 'this' reference is passed from the parameterless constructor to the parameterized constructor.

○ קיימת כפילות קוד בפעולות הבונות

○ מילת המפתח `this`, יכולה לשמש גם לזימון של בנאי אחר במחלקה

○ הבנאי המזומן עם מילת המפתח `this` נכתב לאחר נקודותיים ולפני גוף הפעולה הבונה, ומבוצע לפנייה

```

class Clock
{
    private int minutes, hours;

    public Clock():this(0,0)
    {
        Console.WriteLine("Inside Clock()");
    }

    public Clock(int hours, int minutes)
    {
        Console.WriteLine("Inside1 Clock(hours,minutes)");
        Console.WriteLine("this1 = " + this);
        this.hours = hours;
        this.minutes = minutes;
        Console.WriteLine("this2 = " + this);
        Console.WriteLine(":side1 Clock(hours,minutes)");
    }

    public override string ToString() {...}
}

```

נוסיף הדפסות כדי להבין את סדר ביצוע ההוראות

```
Clock c = new Clock();
```

```

Inside1 Clock(hours,minutes)
this1 = 00:00
this2 = 00:00
Inside1 Clock(hours,minutes)
Inside Clock()

```

```

class Clock
{
    private int minutes, hours;

    public Clock():this(0,0)
    {
        Console.WriteLine("Inside Clock()");
    }

    public Clock(int hours, int minutes)
    {
        Console.WriteLine("Inside1 Clock(hours,minutes)");
        Console.WriteLine("this1 = " + this);
        this.hours = hours;
        this.minutes = minutes;
        Console.WriteLine("this2 = " + this);
        Console.WriteLine(":side1 Clock(hours,minutes)");
    }

    public override string ToString() {...}
}

```

נוסיף הדפסות כדי להבין את סדר ביצוע ההוראות

```
Clock c = new Clock();
```

```

Inside1 Clock(hours,minutes)
this1 = 00:00
this2 = 00:00
Inside1 Clock(hours,minutes)
Inside Clock()

```

```

class Clock
{
    private int minutes, hours;

    public Clock():this(0,0)
    {
        Console.WriteLine("Inside Clock()");
    }

    public Clock(int hours, int minutes)
    {
        Console.WriteLine("Inside1 Clock(hours,minutes)");
        Console.WriteLine("this1 = " + this);
        this.hours = hours;
        this.minutes = minutes;
        Console.WriteLine("this2 = " + this);
        Console.WriteLine(":side1 Clock(hours,minutes)");
    }

    public override string ToString() {...}
}

```

נוסיף הדפסות כדי להבין את סדר ביצוע ההוראות

מההדפסה אנו למדים כי הפעולה הבונה השנייה מתבצעת לפני יתר פעולות הפעולה הבונה הראשונה

יש להבדיל בין `this` שיש בו את ההפניה לעצם הנוכחי ובין הפעולה `this()` המזמנת פעולה בונה אחרת במחלקה

```
Clock c = new Clock();
```

```

Inside1 Clock(hours,minutes)
this1 = 00:00
this2 = 00:00
Inside1 Clock(hours,minutes)
Inside Clock()

```

בדיקת תקינות בפעולה הבונה

○ נרצה למנוע יצירת שעון עם ערכים שאינם תקינים:

```
Clock c = new Clock(30, 30);
```

○ כיוון שפעולות הקביעה שלנו כבר בודקות תקינות, נוכל להשתמש בהן בפעולה הבונה:

```
public Clock(int hours, int minutes)
{
    SetHours(hours);
    SetMinutes(minutes);
}
```


מדוע צריך גם פעולות בונות וגם פעולות קובעות?

הרי תפקיד שתיהן לשים ערכים בעצם?

○ לא מדוייק. תפקידה העיקרי של **הפעולה הבונה** הוא לדאוג לשטחי זיכרון מתאימים ולקבלת ההפנייה אליהם. היא שמה ערכים בתכונות העצם אך ורק ברגע יצירתו

○ **פעולות קובעות** מופעלות על מנת לשנות ערכים של עצם **שכבר נבנה**

פעולות בונות ופעולות קובעות מבצעות פעולת השמה ושינוי ערכי תכונות, אך הן מופעלות בזמנים אחרים במהלך חיי העצם

השמת משתנים

ההבדל בין השמת משתנה פרימיטיבי למשתנה הפניה

```
int x = 3;
```

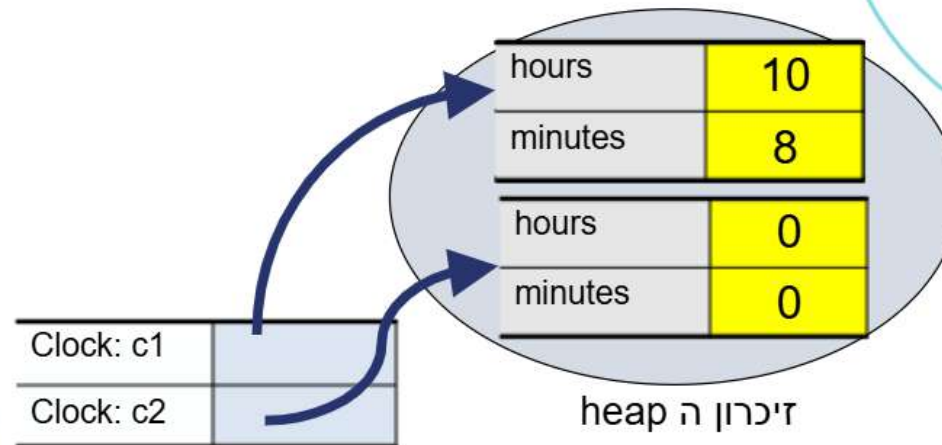
```
int y = 5;
```

```
x = y;
```

int: x	5
int: y	5

```
Clock c1 = new Clock();  
c1.SetHours(10);  
c1.SetMinutes(8);
```

```
Clock c2 = new Clock();
```



השמת משתנים

ההבדל בין השמת משתנה פרימיטיבי למשתנה הפניה

```
int x = 3;
```

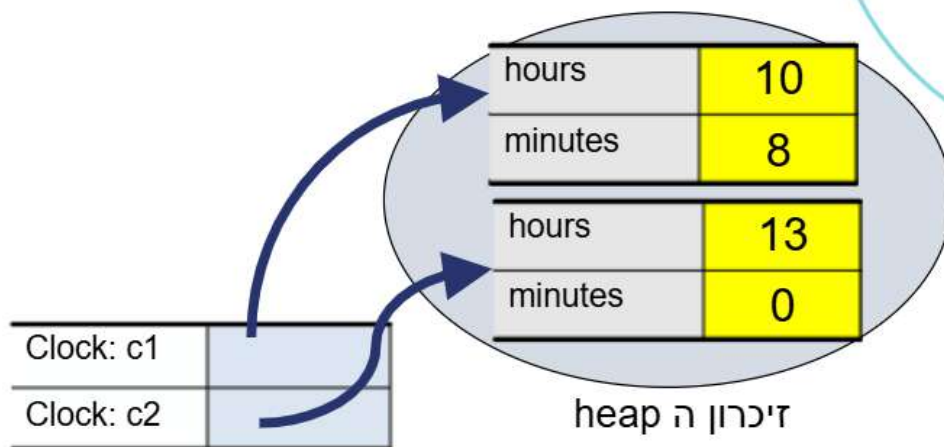
```
int y = 5;
```

```
x = y;
```

int: x	5
int: y	5

```
Clock c1 = new Clock();  
c1.SetHours(10);  
c1.SetMinutes(8);
```

```
Clock c2 = new Clock();  
c2.SetHours(13);
```



השמת משתנים

ההבדל בין השמת משתנה פרימיטיבי למשתנה הפניה

```
int x = 3;
```

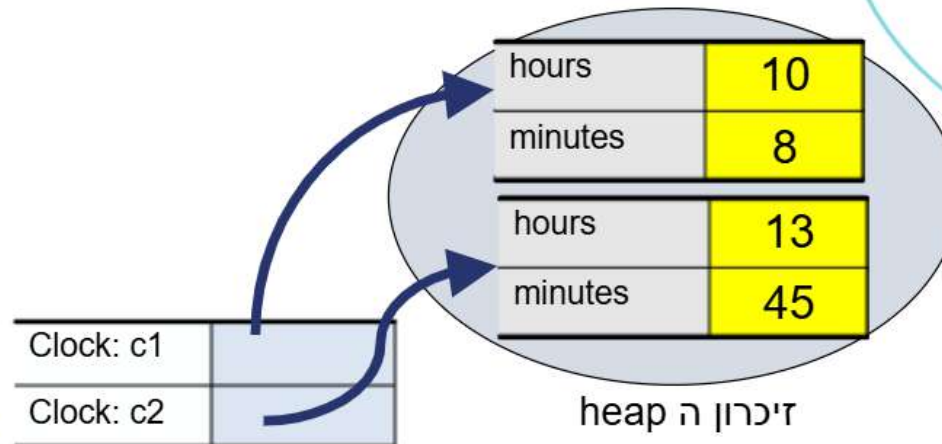
```
int y = 5;
```

```
x = y;
```

int: x	5
int: y	5

```
Clock c1 = new Clock();  
c1.SetHours(10);  
c1.SetMinutes(8);
```

```
Clock c2 = new Clock();  
c2.SetHours(13);  
c2.SetMinutes(45);
```



השמת משתנים

ההבדל בין השמת משתנה פרימיטיבי למשתנה הפניה

```
int x = 3;  
int y = 5;
```

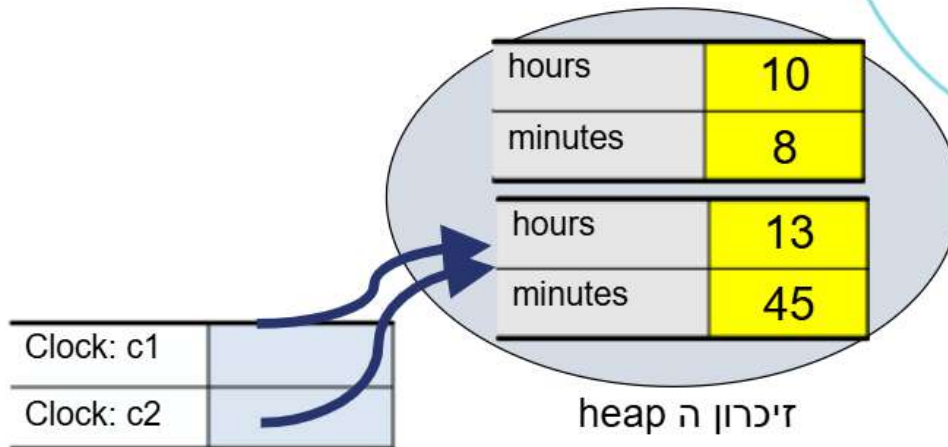
```
x = y;
```

int: x	5
int: y	5

```
Clock c1 = new Clock();  
c1.SetHours(10);  
c1.SetMinutes(8);
```

```
Clock c2 = new Clock();  
c2.SetHours(13);  
c2.SetMinutes(45);
```

```
c1 = c2;
```



השמת משתנים

ההבדל בין השמת משתנה פרימיטיבי למשתנה הפניה

```
int x = 3;
```

```
int y = 5;
```

```
x = y;
```

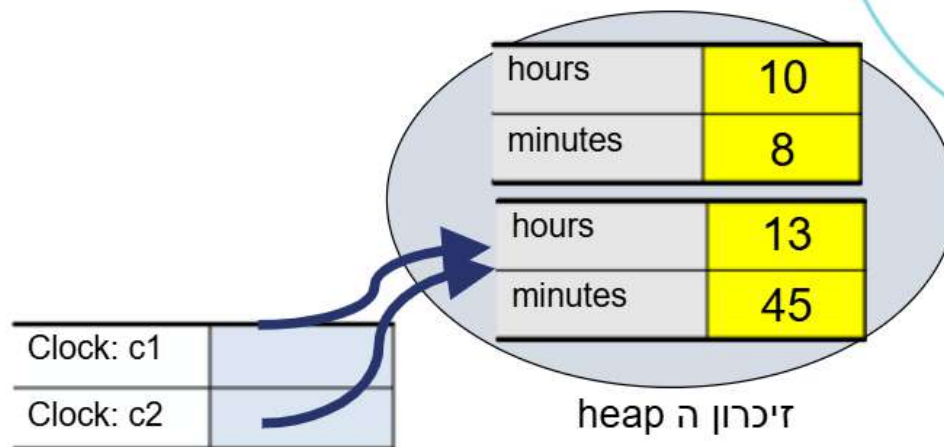
int: x	5
int: y	5

השמה של משתנה הפניה
שונה מהשמה של משתנה פרימיטיבי!

```
Clock c1 = new Clock();  
c1.SetHours(10);  
c1.SetMinutes(8);
```

```
Clock c2 = new Clock();  
c2.SetHours(13);  
c2.SetMinutes(45);
```

```
c1 = c2;
```



השמת משתנים

ההבדל בין השמת משתנה פרימיטיבי למשתנה הפניה

```
int x = 3;  
int y = 5;
```

`x = y;`

int: x	5
int: y	5

השמה של משתנה הפניה
שונה מהשמה של משתנה פרימיטיבי!

השמה הפניה היא ההעתקת ההפניה
למשתנה האחר (ולא העתקת העצם!)

באופן זה נקבל שתי הפניות
שונות לאותו עצם!

Clock: c1	
Clock: c2	

hours	10
minutes	8
hours	13
minutes	45

זיכרון ה heap

```
Clock c1 = new Clock();  
c1.SetHours(10);  
c1.SetMinutes(8);
```

```
Clock c2 = new Clock();  
c2.SetHours(13);  
c2.SetMinutes(45);
```

`c1 = c2;`



שכפול עצם – כיצד?

○ ראינו שלא ניתן לשכפל עצם באמצעות השמה של משתנה ההפניה
○ איך בכל זאת נעשה זאת?

○ דרך אחת היא בניית עצם חדש כשהפרמטרים הם ערכי תכונות העצם, אותו רוצים לשכפל, באמצעות פעולות אחזור התכונות:

```
Clock c1 = new Clock(6, 15);  
Clock c2 = new Clock(c1.GetHours(), c1.GetMinutes());
```

○ אך עדיף לספק במחלקה פעולה בונה מעתיקה – שמטרתה לבנות עצם חדש עם אותם ערכים

פעולה בונה מעתיקה – Copy Constructor

- פעולה בונה מעתיקה היא פעולה בונה שהפרמטר שהיא מקבלת הוא **עצם אחר מאותו הטיפוס**
- מטרתה ליצור עצם **חדש נוסף**, הזהה בערכיו לעצם שהתקבל כפרמטר



```
public Clock(Clock other)
{
    this.hours = other.hours;
    this.minutes = other.minutes;
}
```

בפעולה בונה מעתיקה ניתן לגשת לכל התכונות, גם הפרטיות, של שני העצמים. הם שייכים לאותה המחלקה

אין צורך להשתמש בפעולות קובעות בפעולה בונה מעתיקה, משום שבעצם `other` יש תמיד ערכים תקינים!


```
class Clock
```

```
{
```

```
    private int hours, minutes;
```

```
    public Clock(int hours, int minutes)
```

```
    {
```

```
        SetHours(hours);
```

```
        SetMinutes(minutes);
```

```
    }
```

```
    public Clock(Clock other)
```

```
    {
```

```
        this.hours = other.hours;
```

```
        this.minutes = other.minutes;
```

```
    }
```

```
    public int GetHours() {...}
```

```
    public int GetMinutes() {...}
```

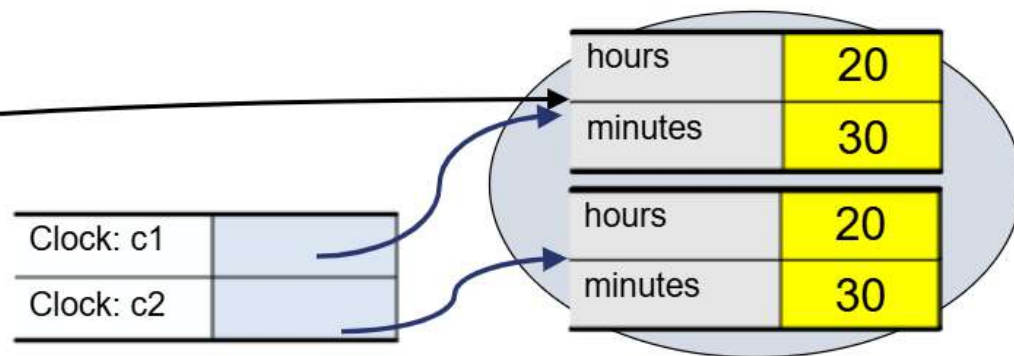
```
    public bool SetMinutes(int minutes) {...}
```

```
    public bool SetHours(int hours) {...}
```

```
    public override string ToString() {...}
```

```
}
```

פעולה בונה מעתיקה דוגמה



```
static void Main(string[] args)
```

```
{
```

```
    Clock c1 = new Clock(20, 30);
```

```
    Clock c2 = new Clock(c1);
```

```
    c1.SetHours(21);
```

```
}
```



```
class Clock
{
    private int hours, minutes;

    public Clock():this(0,0) {...}
    public Clock(int hours, int minutes) {...}
    public Clock(Clock other) {...}
    public int GetHours() {...}
    public int GetMinutes() {...}
    public bool SetMinutes(int minutes) {...}
    public bool SetHours(int hours) {...}
    public override string ToString() {...}

    public bool IsNight() {...}

    public int MinutesToMidnight() {...}

    public void Add(int minutesToAdd) {...}
}
```

פעולות נוספות במחלקה

- מחלקה היא מבנה לוגי שנכתב למטרה מסוימת
- מעבר לפעולות בונות, פעולת ToString ופעולות אחזור וקביעה, נוסף למחלקה פעולות הקשורות במטרת המחלקה

```

class Clock
{
    private int hours, minutes;

    public Clock():this(0,0) {...}
    public Clock(int hours, int minutes) {...}
    public Clock(Clock other) {...}
    public int GetHours() {...}
    public int GetMinutes() {...}
    public bool SetMinutes(int minutes) {...}
    public bool SetHours(int hours) {...}
    public override string ToString() {...}

    public bool IsNight() {...}

    public int MinutesToMidnight() {...}

    public void Add(int minutesToAdd) {...}

    public bool Equals(Clock other) {...}
}

```

פעולות נוספות במחלקה

- מחלקה היא מבנה לוגי שנכתב למטרה מסוימת
- מעבר לפעולות בונות, פעולת ToString ופעולות אחזור וקביעה, נוסף למחלקה פעולות הקשורות במטרת המחלקה

```

class Clock
{
    private int hours, minutes;

    public Clock():this(0,0) {...}
    public Clock(int hours, int minutes) {...}
    public Clock(Clock other) {...}
    public int GetHours() {...}
    public int GetMinutes() {...}
    public bool SetMinutes(int minutes) {...}
    public bool SetHours(int hours) {...}
    public override string ToString() {...}

    public bool IsNight() {...}

    public int MinutesToMidnight() {...}

    public void Add(int minutesToAdd) {...}

    public bool Equals(Clock other) {...}

    public bool IsEarlierThan(Clock other) {...}
}

```

פעולות נוספות במחלקה

- מחלקה היא מבנה לוגי שנכתב למטרה מסוימת

- מעבר לפעולות בונות, פעולת ToString ופעולות אחזור וקביעה, נוסף למחלקה פעולות הקשורות במטרת המחלקה

פעולות נוספות במחלקה

- פעולות במחלקה אינן מקבלות את העצם כפרמטר, הזימון הוא ע"י ההפניה לעצם נקודה פעולה

- שימו לב להעדר המילה static, כיוון שאלו פעולות המופעלות על עצמים

```
// הפעולה מחזירה אמת אם השעה היא שעת לילה/  
// לילה מוגדר משעה 20:00 בערב ועד 5:00 בבוקר/  
public bool IsNight()  
{  
    return hours < 5 || hours >= 20;  
}
```

```
// הפעולה מחזירה את מספר הדקות/  
// שנותרו עד חצות הלילה/  
public int MinutesToMidnight()  
{  
    int minutesToMidnight = (24 * 60) - (hours * 60 + minutes);  
    return minutesToMidnight;  
}
```

```
Clock c1 = new Clock(23, 30);  
Console.WriteLine(c1 + " - Is Night? " + c1.IsNight());  
Console.WriteLine(c1 + " - MinutesToMidnight: " + c1.MinutesToMidnight());  
  
Clock c2 = new Clock(12, 30);  
Console.WriteLine(c2 + " - Is Night? " + c2.IsNight());  
Console.WriteLine(c2 + " - MinutesToMidnight: " + c2.MinutesToMidnight());
```

```
23:30 - Is Night? True  
23:30 - MinutesToMidnight: 30  
12:30 - Is Night? False  
12:30 - MinutesToMidnight: 690
```



```

public void Add(int minutesToAdd)
{
    minutes += minutesToAdd;
    NormalizeTime();
}
public void Subtract(int minutesToSubtract)
{
    minutes -= minutesToSubtract;
    NormalizeTime();
}
private void NormalizeTime()
{
    while (minutes < 0)
    {
        minutes += 60;
        hours--;
    }
    while (minutes >= 60)
    {
        minutes -= 60;
        hours++;
    }
    hours = (hours % 24 + 24) % 24;
}

```

פעולות נוספות במחלקה

- פעולות במחלקה יכולות גם להיות פרטיות, אך אם פעולה הוגדרה כפרטית, ניתן להשתמש בה רק בתוך המחלקה
- כאן, הפעולות המוסיפות דקות לשעון (**Add**) ומפחיתות דקות מהשעון (**Subtract**), משתמשות בפעולה פרטית (**NormalizeTime**), המנרמלת את הזמן בשעון, כך שערכי התכונות ישארו בטווח המותר

```

Clock c1 = new Clock(23, 30);
Console.WriteLine(c1);
c1.Add(35);
Console.WriteLine(c1);
c1.Subtract(90);
Console.WriteLine(c1);

```

23:30
00:05
22:35

פעולות נוספות במחלקה

```
// הפעולה מקבלת שעון/  
// הפעולה מחזירה אמת אם השעונים מראים אותה שעה ושקר אחרת/  
public bool Equals(Clock other)  
{  
  
}  
  
// הפעולה מקבלת שעון/  
// הפעולה מחזירה אמת אם השעון הנוכחי מוקדם/  
// מהשעון המועבר כפרמטר ושקר אחרת - באותה 'ממה'/  
public bool IsEarlierThan(Clock other)  
{  
  
}
```

```
// הפעולה מקבלת שעון/  
// הפעולה מחזירה אמת אם השעונים מראים אותה שעה ושקר אחרת/  
public bool Equals(Clock other)  
{  
    return hours == other.hours &&  
           minutes == other.minutes;  
}  
  
// הפעולה מקבלת שעון/  
// הפעולה מחזירה אמת אם השעון הנוכחי מוקדם/  
// מהשעון המועבר כפרמטר ושקר אחרת - באותה יממה/  
public bool IsEarlierThan(Clock other)  
{  
    if (hours < other.hours)  
        return true;  
    else if (hours == other.hours)  
        return minutes < other.minutes;  
    return false;  
}
```

פעולות נוספות במחלקה

- פעולות המחלקה יכולות לקבל (הפניות ל) עצמים כפרמטר
- כיוון שאנו בתוך המחלקה, יש גישה ישירה לתכונות העצם שהועבר כפרמטר

פעולות נוספות במחלקה

הפעולה מקבלת שעון/

הפעולה מחזירה אמת אם השעונים מראים אותה שעה ושקר אחרת/

```
public bool Equals(Clock other)
```

```
{  
    return hours == other.hours &&  
           minutes == other.minutes;  
}
```

הפעולה מקבלת שעון/

הפעולה מחזירה אמת אם השעון הנוכחי מוקדם/

השעון המועבר כפרמטר ושקר אחרת - באותה יממה/

```
public bool IsEarlierThan(Clock other)
```

```
{  
    if (hours < other.hours)  
        return true;  
    else if (hours == other.hours)  
        return minutes < other.minutes;  
    return false;  
}
```

```
Clock c1 = new Clock(23, 30);  
Clock c2 = new Clock(23, 40);  
Clock c3 = new Clock(22, 50);  
Clock c4 = new Clock(23, 30);  
Console.WriteLine(c1 + " Equals " + c2 + " = " + c1.Equals(c2));  
Console.WriteLine(c1 + " Equals " + c3 + " = " + c1.Equals(c3));  
Console.WriteLine(c1 + " Equals " + c4 + " = " + c1.Equals(c4));  
Console.WriteLine(c1 + " IsEarlierThan " + c2 + " = " +  
c1.IsEarlierThan(c2));  
Console.WriteLine(c1 + " IsEarlierThan " + c3 + " = " +  
c1.IsEarlierThan(c3));  
Console.WriteLine(c1 + " IsEarlierThan " + c4 + " = " +  
c1.IsEarlierThan(c4));
```

```
23:30 Equals 23:40 = False  
23:30 Equals 22:50 = False  
23:30 Equals 23:30 = True  
23:30 IsEarlierThan 23:40 = True  
23:30 IsEarlierThan 22:50 = False  
23:30 IsEarlierThan 23:30 = False
```

פעולות נוספות במחלקה

הפעולה מקבלת שעון/

הפעולה מחזירה אמת אם השעונים מראים אותה שעה ושקר אחרת/

```
public bool Equals(Clock other)
{
    return hours == other.hours &&
           minutes == other.minutes;
}
```

הפעולה מקבלת שעון/

הפעולה מחזירה אמת אם השעון הנוכחי מוקדם/

השעון המועבר כפרמטר ושקר אחרת - באותה יממה/

```
public bool IsEarlierThan(Clock other)
{
    if (hours < other.hours)
        return true;
    else if (hours == other.hours)
        return minutes < other.minutes;
    return false;
}
```

פעולות אלו מקבלות כפרמטר עצם מטיפוס
שעון. מה באמת מועבר?

```
Clock c1 = new Clock(23, 30);
Clock c2 = new Clock(23, 40);
Clock c3 = new Clock(22, 50);
Clock c4 = new Clock(23, 30);
Console.WriteLine(c1 + " Equals " + c2 + " = " + c1.Equals(c2));
Console.WriteLine(c1 + " Equals " + c3 + " = " + c1.Equals(c3));
Console.WriteLine(c1 + " Equals " + c4 + " = " + c1.Equals(c4));
Console.WriteLine(c1 + " IsEarlierThan " + c2 + " = " +
c1.IsEarlierThan(c2));
Console.WriteLine(c1 + " IsEarlierThan " + c3 + " = " +
c1.IsEarlierThan(c3));
Console.WriteLine(c1 + " IsEarlierThan " + c4 + " = " +
c1.IsEarlierThan(c4));
```

```
23:30 Equals 23:40 = False
23:30 Equals 22:50 = False
23:30 Equals 23:30 = True
23:30 IsEarlierThan 23:40 = True
23:30 IsEarlierThan 22:50 = False
23:30 IsEarlierThan 23:30 = False
```




העברת עצמים לפעולות

העברה by value

כאשר פעולה מקבלת משתנה פרימיטיבי, היא מקבלת העתק שלו, ולכן אינה משנה את הערך המקורי

העברה by value

כאשר פעולה מקבלת עצם, היא מקבלת העתק של ההפניה אליו, שינוי של תכונות העצם יתבצע בזיכרון וההפניה המקורית (הכתובת) לא תשתנה

*העברה by reference

ב C# ניתן להעביר גם פרמטרים עם המילים השמורות ref/in/out המקבלות גישה למשתנה אשר מועבר כפרמטר, ומשנות את תוכנו



העברת עצמים לפעולות

העברה by value

כאשר פעולה מקבלת משתנה פרימיטיבי, היא מקבלת העתק שלו, ולכן אינה משנה את הערך המקורי

העברה by value

כאשר פעולה מקבלת עצם, היא מקבלת העתק של ההפניה אליו, שינוי של תכונות העצם יתבצע בזיכרון וההפניה המקורית (הכתובת) לא תשתנה

*העברה by reference

ב C# ניתן להעביר גם פרמטרים עם המילים השמורות ref/in/out המקבלות גישה למשתנה אשר מועבר כפרמטר, ומשנות את תוכנו

העברת עצמים לפעולות

```
static void Foo(int x, Clock c)
{
    x = 5;
    c.SetMinutes(59);
}

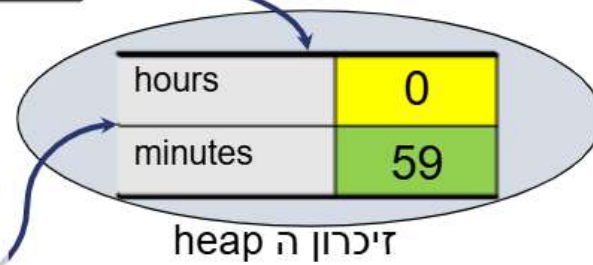
static void Main(string[] args)
{
    int num = 2;
    Clock myClock = new Clock();

    Foo(num, myClock);

    Console.WriteLine("num=" + num);
    Console.WriteLine(myClock);
}
```

Foo	
int: x	5
Clock: c	

Main	
int: num	2
Clock: myClock	



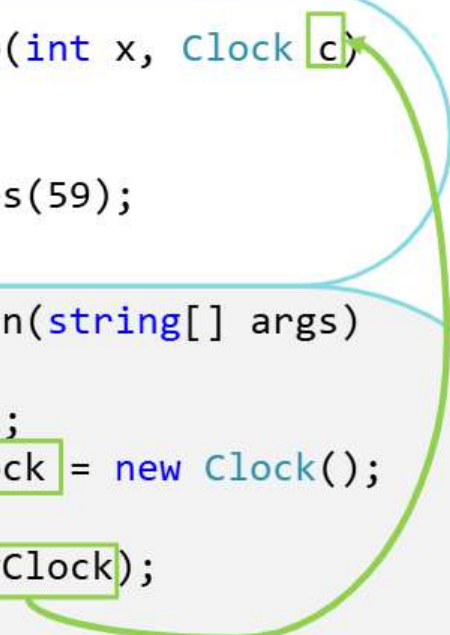
העברת עצמים לפעולות

```
static void Foo(int x, Clock c)
{
    x = 5;
    c.SetMinutes(59);
}

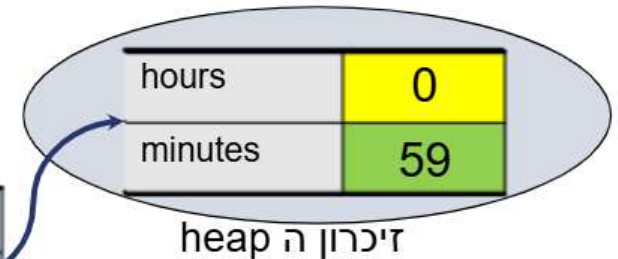
static void Main(string[] args)
{
    int num = 2;
    Clock myClock = new Clock();

    Foo(num, myClock);

    Console.WriteLine("num=" + num);
    Console.WriteLine(myClock);
}
```



Main	
int: num	2
Clock: myClock	



העברת עצמים לפעולות

העברה by value

כאשר פעולה מקבלת עצם, היא מקבלת העתק של ההפניה אליו, שינוי של תכונות העצם יתבצע בזיכרון וההפניה המקורית (הכתובת) לא תשתנה

```
static void Foo(int x, Clock c)
{
    x = 5;
    c.SetMinutes(59);
}
```

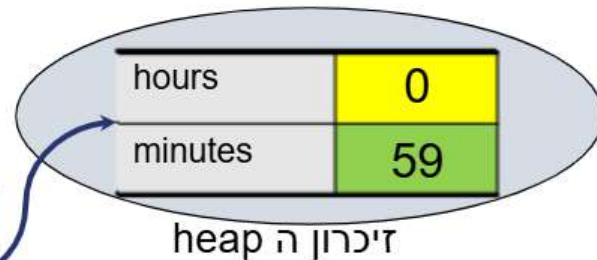
```
static void Main(string[] args)
{
    int num = 2;
    Clock myClock = new Clock();

    Foo(num, myClock);

    Console.WriteLine("num=" + num);
    Console.WriteLine(myClock);
}
```

num=2
00:59

Main	
int: num	2
Clock: myClock	



מערך עצמים

מערך הוא אוסף סדור של **משתנים מטיפוס מסויים**, בגודל קבוע
בניית מערך מטיפוס Clock בגודל 5:

```
Clock[] arr = new Clock[5];
```



המערך נוצר, וב arr יש הפניה אליו. בכל התאים יש null, כיוון שמערך נוצר עם ערכי ברירת מחדל



מערך עצמים – תא במערך הוא משתנה הפניה:

```
Clock[] arr = new Clock[5];  
arr[0] = new Clock(9,30);  
Console.WriteLine(arr[0].GetHours()); // 9  
arr[0].SetMinutes(15);  
Console.WriteLine(arr[0]); // 09:15
```

זימון פעולה בונה והשמת
ההפניה בתא במערך



מערך עצמים – תא במערך הוא משתנה הפניה:

```
Clock[] arr = new Clock[5];  
arr[0] = new Clock(9,30);  
Console.WriteLine(arr[0].GetHours()); // 9  
arr[0].SetMinutes(15);  
Console.WriteLine(arr[0]); // 09:15
```

זימון פעולה בונה והשמת
ההפניה בתא במערך



מערך עצמים – תא במערך הוא משתנה הפניה:

```
Clock[] arr = new Clock[5];  
arr[0] = new Clock(9,30);  
Console.WriteLine(arr[0].GetHours()); // 9  
arr[0].SetMinutes(15);  
Console.WriteLine(arr[0]); // 09:15
```

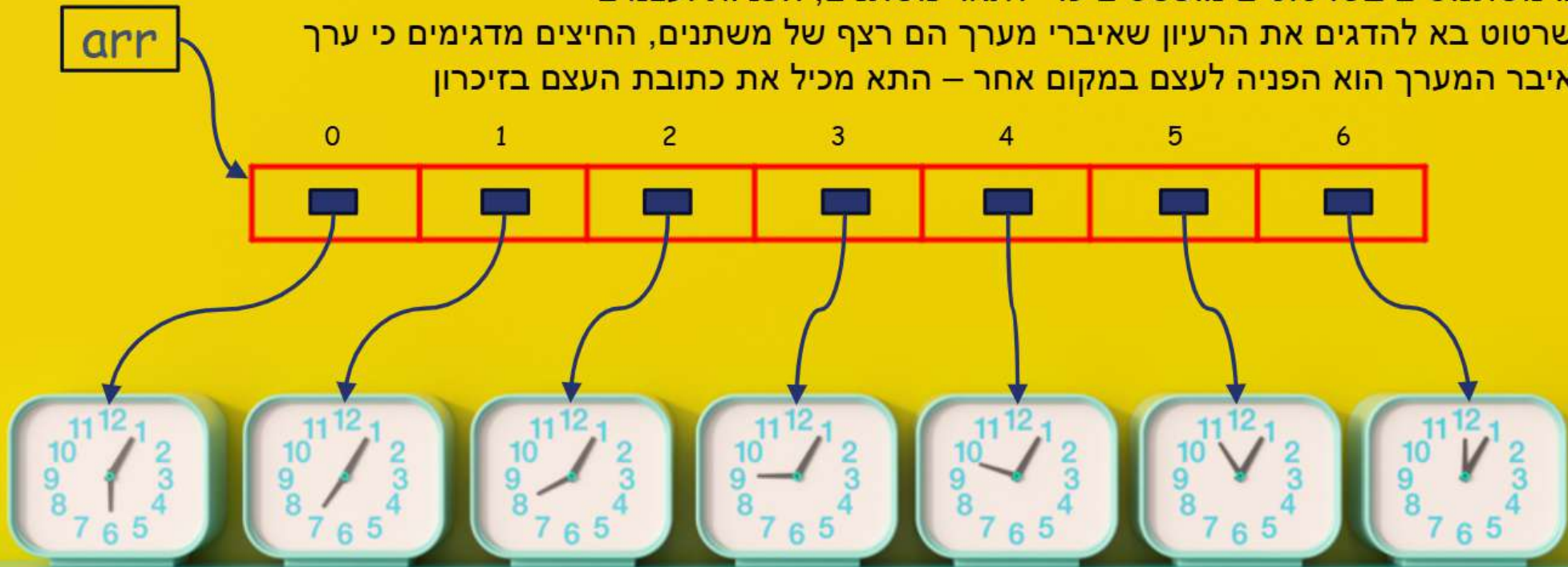
זימון פעולה בונה והשמת
ההפניה בתא במערך

זימון פעולה על העצם,
באמצעות סימן הנקודה על
תא במערך, המכיל הפניה
לעצם



מערך עצמים – שרטוט:

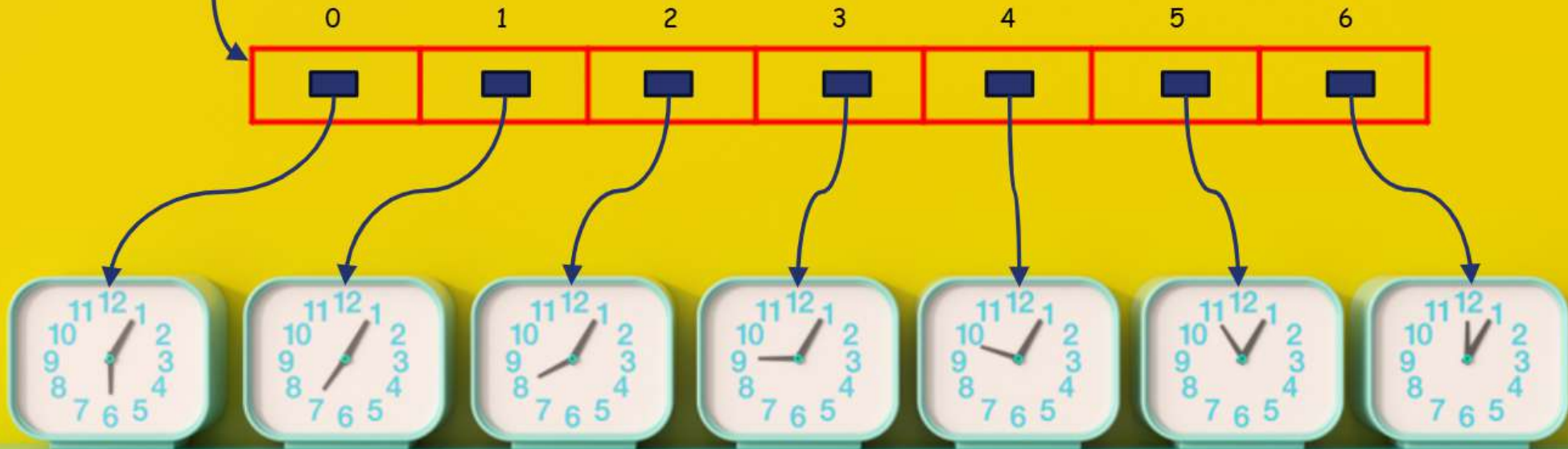
אנו משתמשים בשרטוטים מופשטים כדי לתאר משתנים, הפניות ועצמים. השרטוט בא להדגים את הרעיון שאיברי מערך הם רצף של משתנים, החיצים מדגימים כי ערך האיבר המערך הוא הפניה לעצם במקום אחר – התא מכיל את כתובת העצם בזיכרון.



מערך עצמים – שרטוט:

בשפת יום יום נאמר כי arr הוא מערך
שעונים. בפועל הוא הפניה למערך
שאיבריו הם הפניות לעצמים מטיפוס שעון

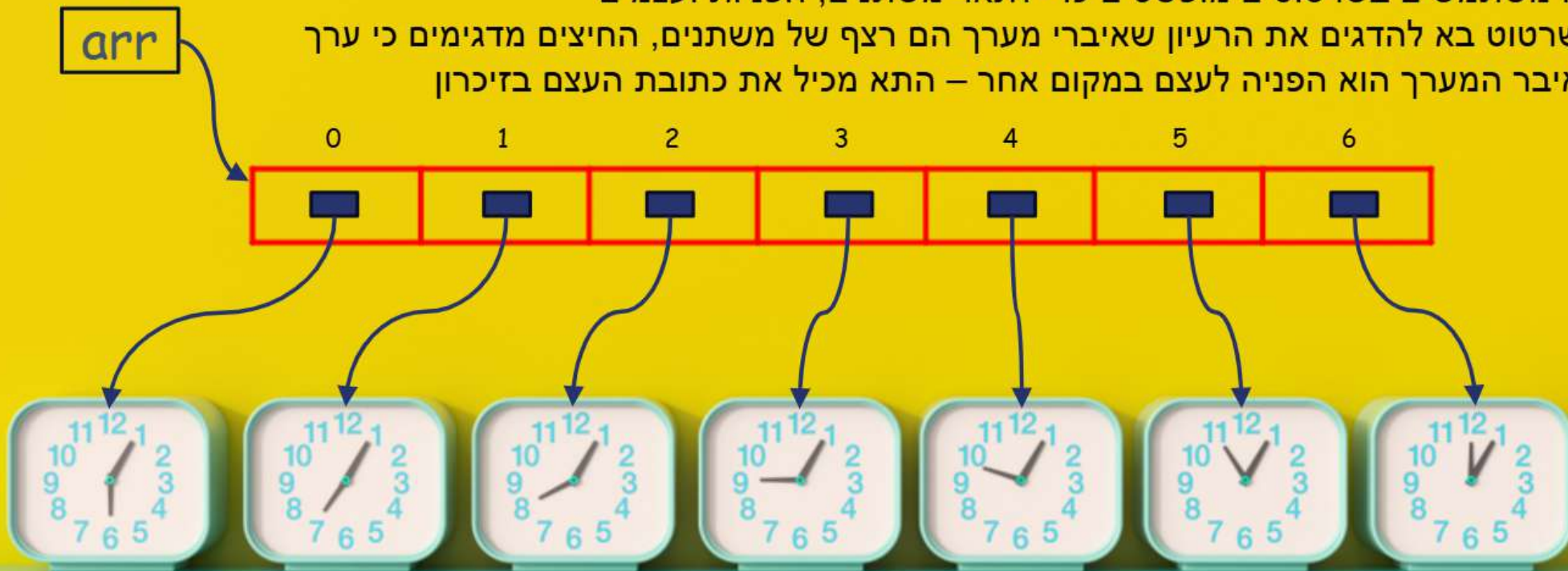
אנו משתמשים בשרטוטים מופשטים כדי לתאר משתנים, הפניות ועצמים
השרטוט בא להדגים את הרעיון שאיברי מערך הם רצף של משתנים, החיצים מדגימים כי ערך
האיבר המערך הוא הפניה לעצם במקום אחר – התא מכיל את כתובת העצם בזיכרון



מערך עצמים – שרטוט:

בשפת יום יום נאמר כי `arr` הוא מערך
שעונים. בפועל הוא הפניה למערך
שאיבריו הם הפניות לעצמים מטיפוס שעון

אנו משתמשים בשרטוטים מופשטים כדי לתאר משתנים, הפניות ועצמים
השרטוט בא להדגים את הרעיון שאיברי מערך הם רצף של משתנים, החיצים מדגימים כי ערך
האיבר המערך הוא הפניה לעצם במקום אחר – התא מכיל את כתובת העצם בזיכרון



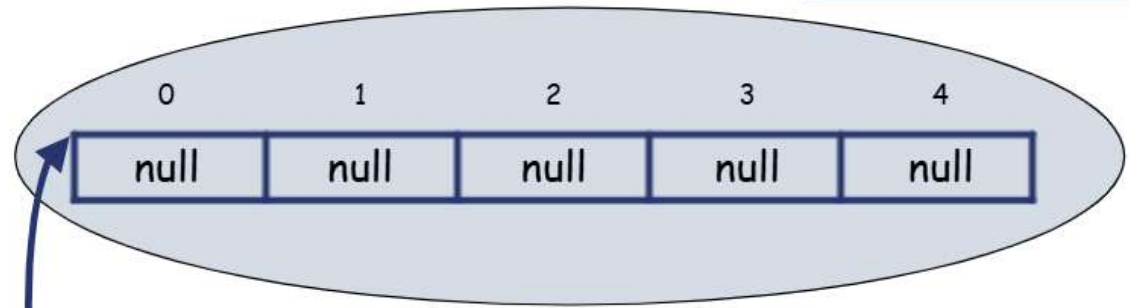
כך לא נראה מערך בזיכרון, ולא עצמים

```

public static void Print(Clock[] arr)
{
    // הפעולה מדפיסה את איברי המערך
    Console.Write("[");
    for (int i = 0; i < arr.Length-1; i++)
        Console.Write(arr[i] + ", ");
    if(arr.Length > 0)
        Console.Write(arr[arr.Length-1]);
    Console.WriteLine("]");
}

// כאן אם התא null מיופס null
public static void PrintAlsoNull(Clock[] arr)
{
    Console.Write("[");
    for (int i = 0; i < arr.Length - 1; i++)
    {
        if(arr[i] == null)
            Console.Write("null, ");
        else
            Console.Write(arr[i] + ", ");
    }
    if (arr.Length > 0)
    {
        if (arr[arr.Length - 1] == null)
            Console.Write("null");
        else
            Console.Write(arr[arr.Length - 1]);
    }
    Console.WriteLine("]");
}

```



```

static void Main(string[] args)
{
    Clock[] arr = new Clock[5];
    Print(arr);
    PrintAlsoNull(arr);
}

```

```

[, , , , ]
[null, null, null, null, null]

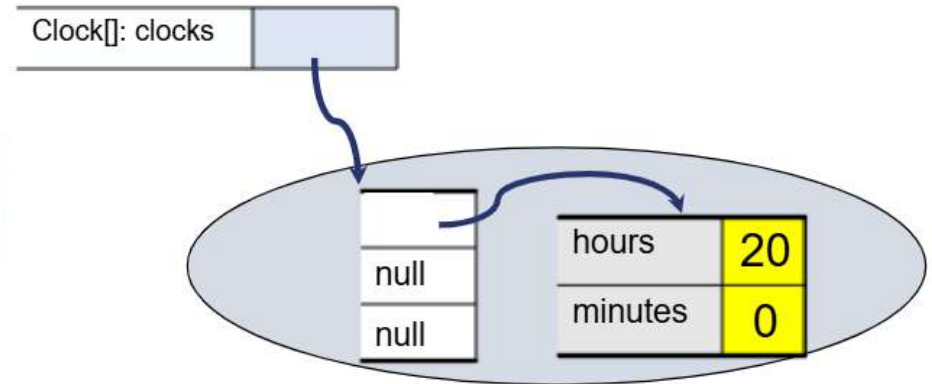
```

מערך של עצמים - בנייה

מערך של עצמים הוא מערך של הפניות. כאשר המערך נבנה, ערכי התאים הוא null
ניסיון לפנות לעצם שאינו קיים (ההפנייה null) גורמת לשגיאה בזמן הריצה

```
static void Main(string[] args)
{
    Clock[] clocks = new Clock[3];

    clocks[0] = new Clock();
    clocks[0].SetHours(20);
    clocks[1].SetMinutes(40);
}
```

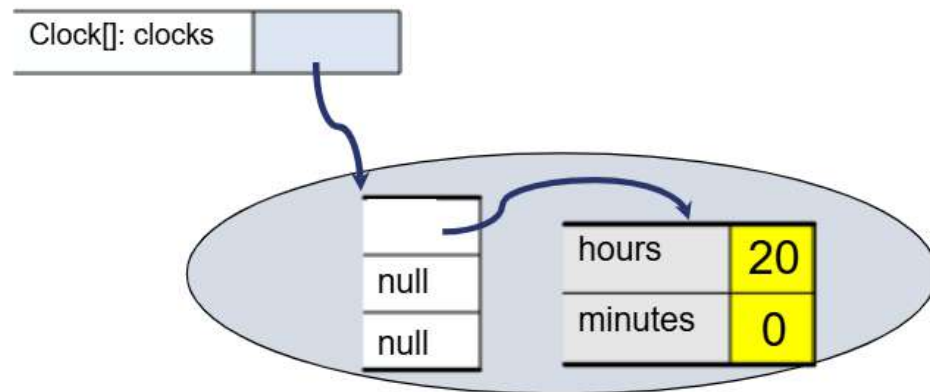


מערך של עצמים - בנייה

מערך של עצמים הוא מערך של הפניות. כאשר המערך נבנה, ערכי התאים הוא null
ניסיון לפנות לעצם שאינו קיים (ההפנייה null) גורמת לשגיאה בזמן הריצה

```
static void Main(string[] args)
{
    Clock[] clocks = new Clock[3];

    clocks[0] = new Clock();
    clocks[0].SetHours(20);
    clocks[1].SetMinutes(40);
}
```



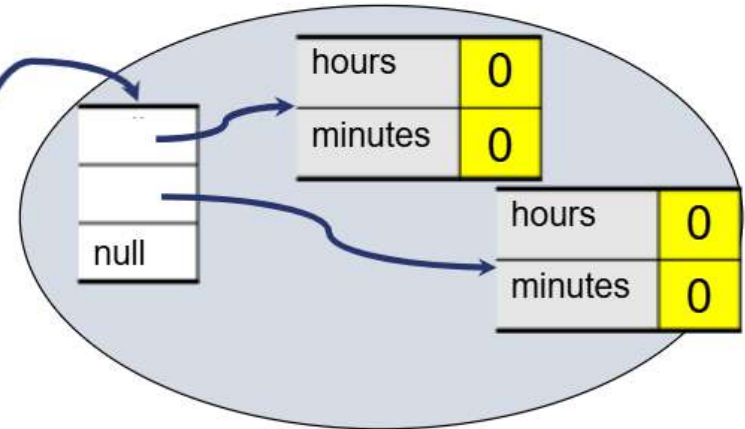
Unhandled Exception: System.NullReferenceException: Object reference not set to an instance of an object.
at clock.Program.Main(String[] args) in C:\Objects and Classes\clock\Program.cs:line 17

מערך של עצמים - בנייה

בעת יצירת מערך של עצמים, כל איברי המערך מקבלים את הערך **null** יש לבצע השמה לתאי המערך של (הפניות ל) עצמים מטיפוס המערך

דוגמה לבניית עצמים בזיכרון והשמה במערך:

```
static void Main(string[] args)
{
    Clock[] clocks = new Clock[3];
    for (int i=0 ; i < clocks.length ; i++ )
    {
        clocks[i] = new Clock();
    }
}
```



זיכרון ה heap

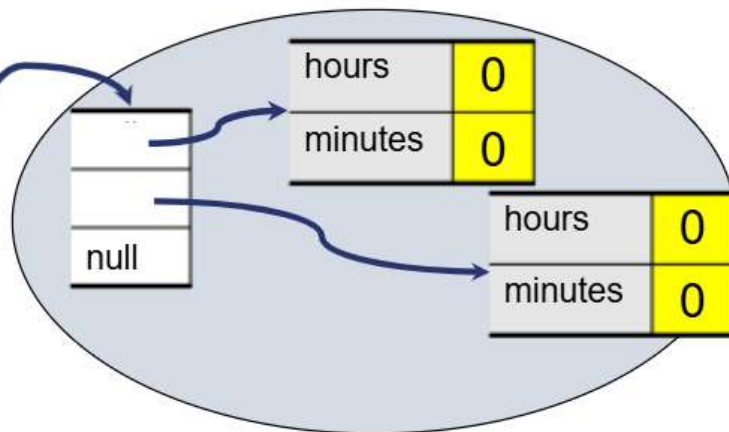
Clock[]: clocks	
int: i	1

מערך של עצמים - בנייה

בעת יצירת מערך של עצמים, כל איברי המערך מקבלים את הערך **null** יש לבצע השמה לתאי המערך של (הפניות ל) עצמים מטיפוס המערך

דוגמה לבניית עצמים בלולאה והשמה במערך:

```
static void Main(string[] args)
{
    Clock[] clocks = new Clock[3];
    for (int i=0 ; i < clocks.length ; i++ )
    {
        clocks[i] = new Clock();
    }
}
```



זיכרון ה heap

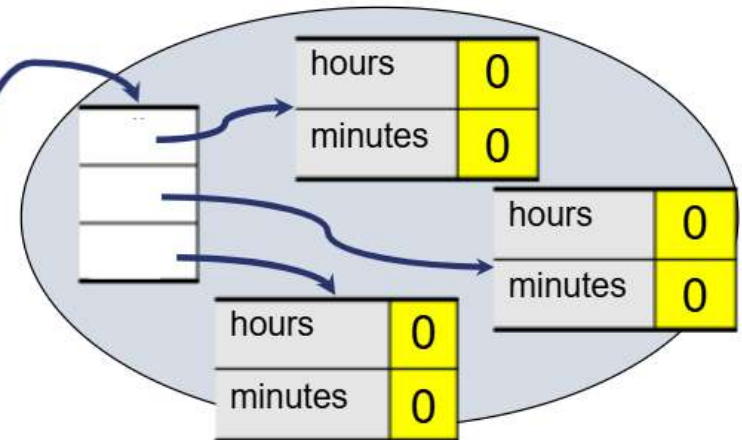
Clock[]: clocks	
int: i	2

מערך של עצמים - בנייה

בעת יצירת מערך של עצמים, כל איברי המערך מקבלים את הערך **null** יש לבצע השמה לתאי המערך של (הפניות ל)עצמים מטיפוס המערך

דוגמה לבניית עצמים בלולאה והשמה במערך:

```
static void Main(string[] args)
{
    Clock[] clocks = new Clock[3];
    for ( int i=0 ; i < clocks.length ; i++ )
    {
        clocks[i] = new Clock();
    }
}
```



זיכרון ה heap

Clock[]: clocks	
int: i	2

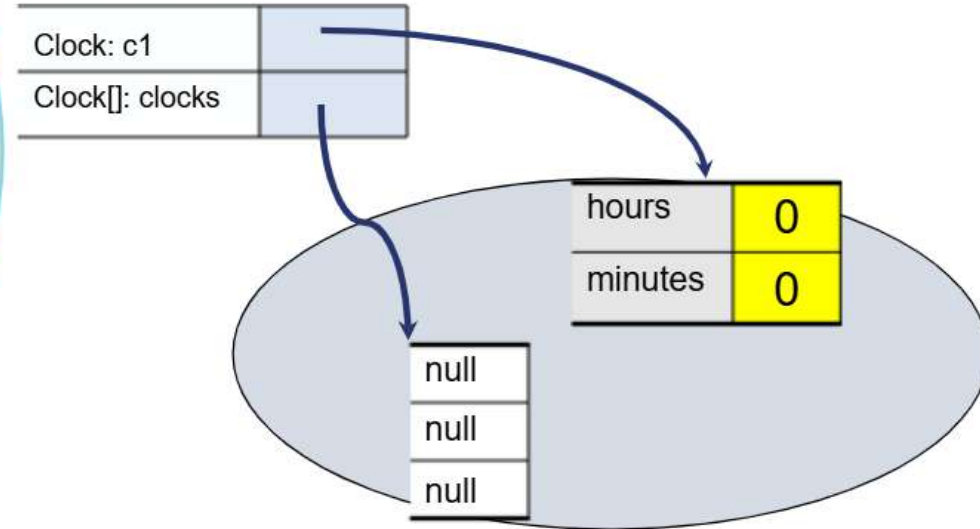
מערך של עצמים - איברים

כל תא במערך הוא משתנה הפניה
ניתן לשים בו (הפניה ל)עצם חדש או הפניה לעצם קיים

```
static void Main(string[] args)
{
    Clock c1 = new Clock();
    Clock[] clocks = new Clock[3];

    c1.SetHour(10);
    c1.SetMinutes(30);

    clocks[0] = c1;
    clocks[1] = new Clock();
    clocks[2] = clocks[1];
}
```



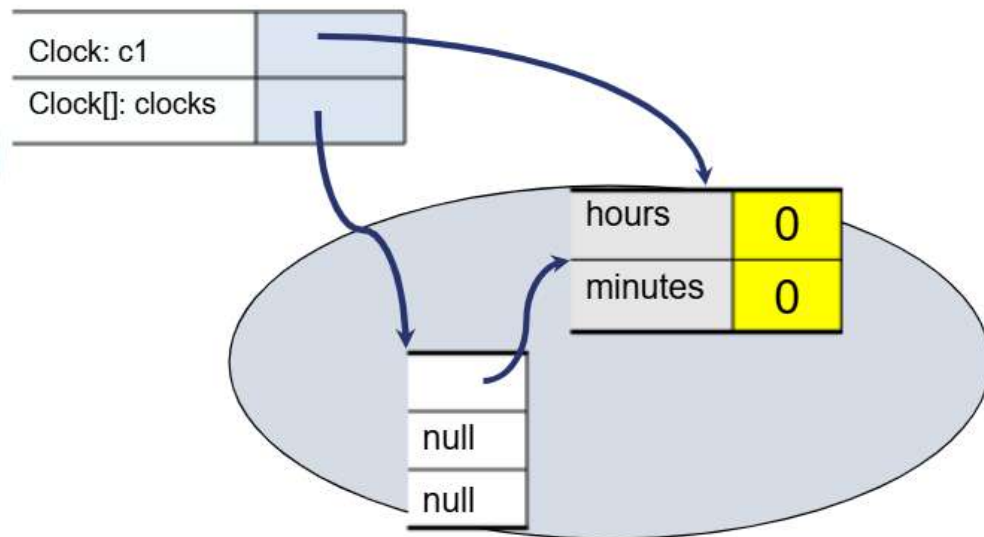
מערך של עצמים - איברים

כל תא במערך הוא משתנה הפניה
ניתן לשים בו (הפניה ל) עצם חדש או הפניה לעצם קיים

```
static void Main(string[] args)
{
    Clock c1 = new Clock();
    Clock[] clocks = new Clock[3];

    c1.SetHour(10);
    c1.SetMinutes(30);

    clocks[0] = c1;
    clocks[1] = new Clock();
    clocks[2] = clocks[1];
}
```



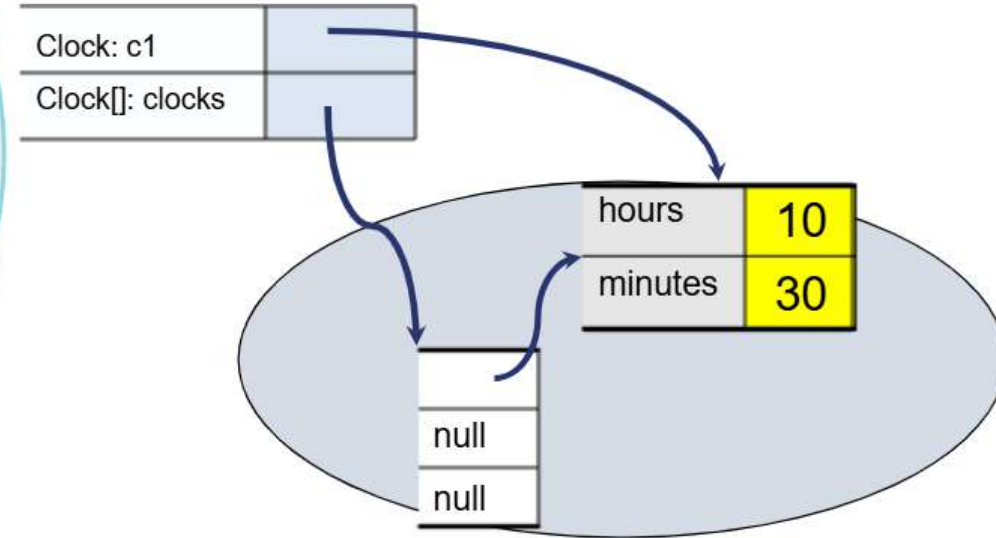
מערך של עצמים - איברים

כל תא במערך הוא משתנה הפניה
ניתן לשים בו (הפניה ל)עצם חדש או הפניה לעצם קיים

```
static void Main(string[] args)
{
    Clock c1 = new Clock();
    Clock[] clocks = new Clock[3];

    c1.SetHour(10);
    c1.SetMinutes(30);

    clocks[0] = c1;
    clocks[1] = new Clock();
    clocks[2] = clocks[1];
}
```



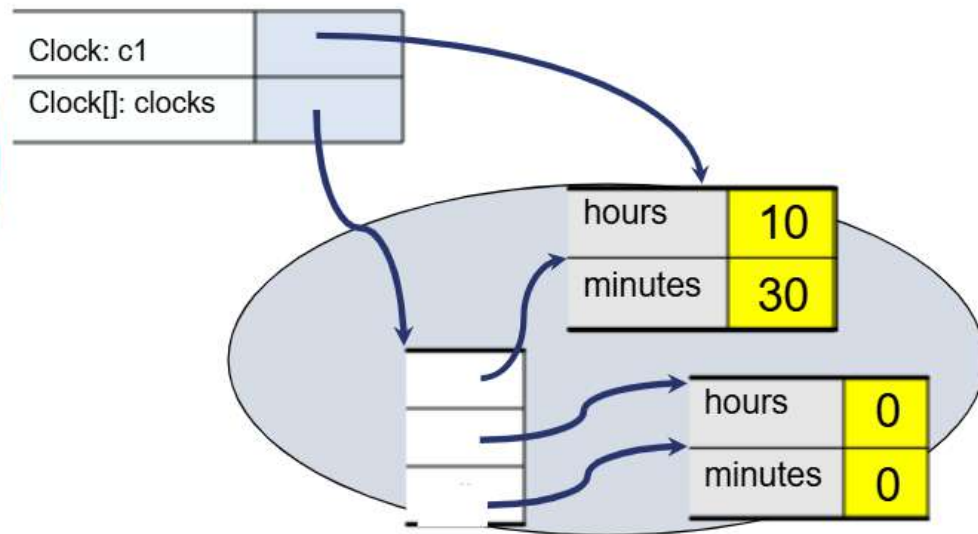
מערך של עצמים - איברים

כל תא במערך הוא משתנה הפניה
ניתן לשים בו (הפניה ל) עצם חדש או הפניה לעצם קיים

```
static void Main(string[] args)
{
    Clock c1 = new Clock();
    Clock[] clocks = new Clock[3];

    c1.SetHour(10);
    c1.SetMinutes(30);

    clocks[0] = c1;
    clocks[1] = new Clock();
    clocks[2] = clocks[1];
}
```



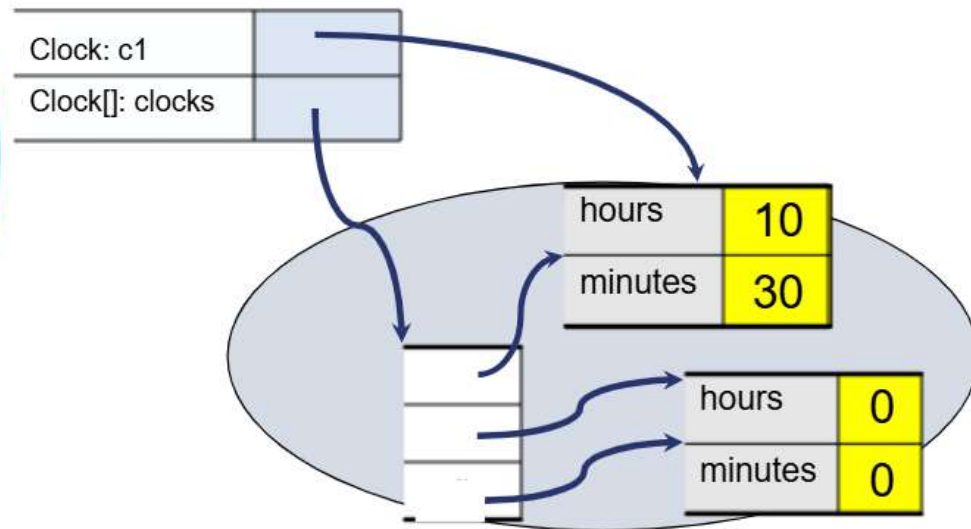
מערך של עצמים - איברים

כל תא במערך הוא משתנה הפניה
ניתן לשים בו (הפניה ל)עצם חדש או הפניה לעצם קיים

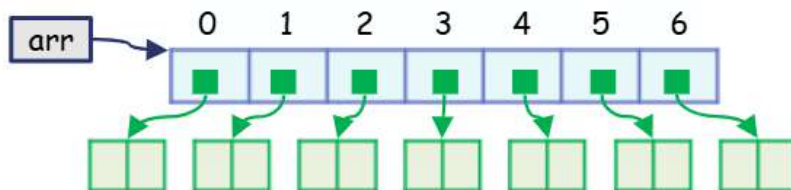
```
static void Main(string[] args)
{
    Clock c1 = new Clock();
    Clock[] clocks = new Clock[3];

    c1.SetHour(10);
    c1.SetMinutes(30);

    clocks[0] = c1;
    clocks[1] = new Clock();
    clocks[2] = clocks[1];
}
```



שני איברים במערך יכולים להכיל הפנייה לאותו העצם



מערך של עצמים - לולאות

ניתן לעבור על מערך עצמים בלולאות for או while, ככל מערך, תוך שימוש במציינים (האינדקסים) של המערך

תא במערך עצמים הוא משתנה המכיל הפניה לעצם ולא את העצם עצמו

```
//פעולה מקבלת אורך מערך רצוי  
//פעולה מחזירה מערך שעונים רנדומלי  
static Clock[] BuildRandomArray(int len)  
{  
    Clock[] arr = new Clock[len];  
    Random random = new Random();  
    for (int i = 0; i < arr.Length; i++)  
    {  
        int hours = random.Next(24);  
        int minutes = random.Next(60);  
        arr[i] = new Clock(hours, minutes);  
    }  
    return arr;  
}
```

```
Clock[] arr1 = BuildRandomArray(7);  
Print(arr1);
```

```
[05:36, 12:42, 18:09, 13:54, 17:47, 06:19, 00:33]
```

מערך של עצמים - לולאות

```
static Clock[] BuildArray(int len,  
                           int hours, int minutes, int add)  
{  
    Clock[] arr = new Clock[len];  
    arr[0] = new Clock(hours, minutes);  
    for (int i = 1; i < arr.Length; i++)  
    {  
        arr[i] = new Clock(arr[i-1]);  
        arr[i].Add(add);  
    }  
    return arr;  
}
```

כתבו פעולה המקבלת אורך מערך רצוי, שעה התחלתית, דקה התחלתית, ומספר דקות add

הפעולה תחזיר מערך שעונים באורך המבוקש, בו השעון הראשון יהיה בעלי ערכי השעות והדקות שהועברו כפרמטר, וכל שעון יהיה מאוחר ב add דקות מקודמו. לדוגמה:

```
Clock[] arr2 = BuildArray(8, 12, 45, 10);  
Print(arr2);
```

```
[12:45, 12:55, 13:05, 13:15, 13:25, 13:35, 13:45, 13:55]
```

מערך של עצמים - לולאות

ניתן להשתמש במשתנה הפניה נוסף לעצם:

//פעולה מקבלת מערך שעונים
//פעולה מחזירה את השעון שמצביע על השעה המוקדמת ביותר

```
static Clock Earliest(Clock[] arr)
```

```
{
```

```
    Clock earliest = arr[0];
```

```
    for (int i = 1; i < arr.Length; i++)
```

```
    {
```

```
        if (arr[i].IsEarlierThan(earliest))  
            earliest = arr[i];
```

```
    }
```

```
    return earliest;
```

```
}
```

העתקת ההפניה
למשתנה מטיפוס
איברי המערך

מזמנים את הפעולה
IsEarlierThan על העצם אליו
מפנה התא הנוכחי במערך

```
static Clock Earliest1(Clock[] arr)
```

```
{
```

```
    Clock earliest = arr[0];
```

```
    for (int i = 1; i < arr.Length; i++)
```

```
    {
```

```
        Clock c = arr[i];
```

```
        if (c.IsEarlierThan(earliest))  
            earliest = c;
```

```
    }
```

```
    return earliest;
```

```
}
```

c

העתקת ההפניה למשתנה, כדי לא להשתמש
בסימן הנקודה על איבר במערך, אלא על
משתנה "רגיל"

מערך של עצמים - לולאות

ניתן לעבור על המערך גם בלולאת while

// הפעולה מקבלת מערך שעונים
// ומספר דקות שיש להוסיף
// הפעולה מוסיפה את הדקות לכל אחד מהשעונים
`static void Add(Clock[] arr, int minutes)`

```
{  
    int i = 0;  
    while(i < arr.Length)  
    {  
        arr[i].Add(minutes);  
        i++;  
    }  
}
```

```
Clock[] arr = BuildArray(7, 22, 45, 15);  
Print(arr);  
Add(arr, 20);
```

פעולה שמשנה איברי מערך אינה צריכה להחזיר ערך. ההפניה למערך שמורה במקום הזימון, השינוי נעשה בערכי המערך עצמו, בזיכרון ה heap

[22:45, 23:00, 23:15, 23:30, 23:45, 00:00, 00:15]

מערך של עצמים - לולאות

ניתן לעבור על המערך גם בלולאת while

```
// הפעולה מקבלת מערך שעונים  
// ומספר דקות שיש להוסיף  
// הפעולה מוסיפה את הדקות לכל אחד מהשעונים  
static void Add(Clock[] arr, int minutes)
```

```
{  
    int i = 0;  
    while(i < arr.Length)  
    {  
        arr[i].Add(minutes);  
        i++;  
    }  
}
```

פעולה שמשנה איברי מערך אינה צריכה להחזיר ערך. ההפניה למערך שמורה במקום הזימון, השינוי נעשה בערכי המערך עצמו, בזיכרון ה heap

```
Clock[] arr = BuildArray(7, 22, 45, 15);  
Print(arr);  
Add(arr, 20);  
Print(arr);
```

```
[22:45, 23:00, 23:15, 23:30, 23:45, 00:00, 00:15]  
[23:05, 23:20, 23:35, 23:50, 00:05, 00:20, 00:35]
```

מערך של עצמים – לולאות – foreach

משמעות – עבור כל אחד מאיברי המערך בצע את גוף הלולאה

הדרך – הגדרת משתנה מטיפוס המערך, שבכל איטרציה, יקבל ערך של איבר במערך - מהאיבר בתא 0 ועד לאחרון.

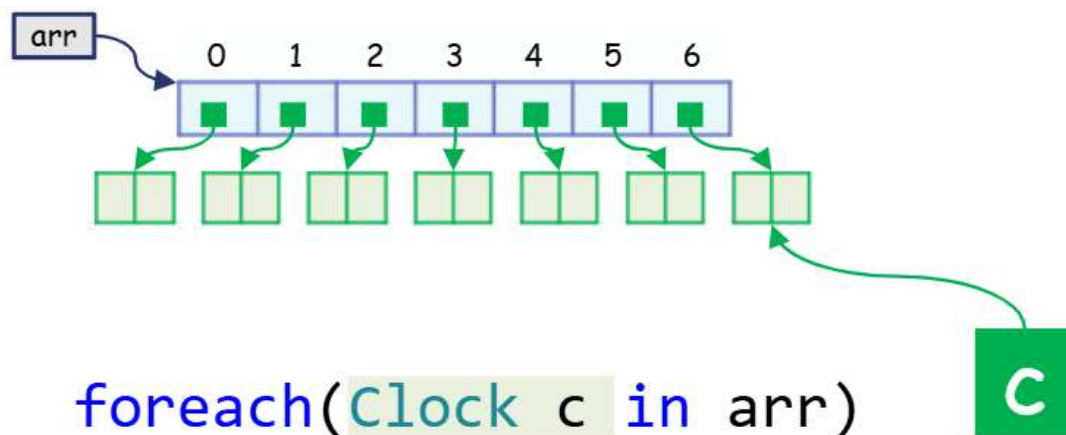
מילה שמורה

הגדרת משתנה מטיפוס המערך

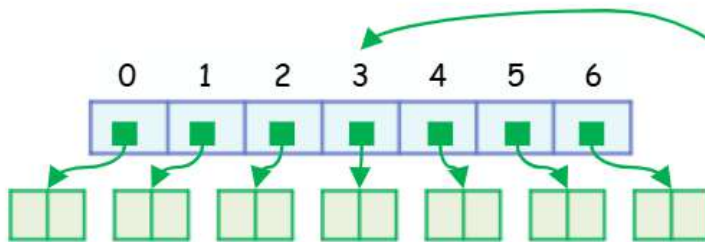
מערך מטיפוס Clock

```
foreach(Clock c in arr)
{
    // do something with c
}
```

מערך של עצמים – לולאות – foreach לולאת



```
foreach(Clock c in arr)
{
    // do something with c
}
```



מערך של עצמים - לולאות

לולאת foreach

```
//פעולה מקבלת מערך שעונים  
//ומספר דקות שיש להפחית  
//הפעולה מפחיתה את הדקות מכל אחד מהשעונים  
static void Subtract (Clock[] arr, int minutes)  
{  
    foreach (Clock c in arr)  
    {  
        c.Subtract(minutesToSubtract);  
    }  
}
```

```
Clock[] arr = BuildRandomArray(7);  
Print(arr);  
Subtract(arr, 120);
```

פעולה שמשנה איברי מערך אינה צריכה להחזיר ערך. ההפניה למערך שמורה במקום הזימון, השינוי נעשה בערכי המערך עצמו, בזיכרון ה heap

[10:07, 00:45, 19:47, 11:38, 02:13, 10:36, 11:08]

מערך של עצמים - לולאות

לולאת foreach

//פעולה מקבלת מערך שעונים
//ומספר דקות שיש להפחית
//הפעולה מפחיתה את הדקות מכל אחד מהשעונים
`static void Subtract (Clock[] arr, int minutes)`
{
 `foreach (Clock c in arr)`
 {
 `c.Subtract(minutesToSubtract);`
 }
}

```
Clock[] arr = BuildRandomArray(7);  
Print(arr);  
Subtract(arr, 120);
```

פעולה שמשנה איברי מערך אינה צריכה להחזיר ערך. ההפניה למערך שמורה במקום הזימון, השינוי נעשה בערכי המערך עצמו, בזיכרון הheap

[10:07, 00:45, 19:47, 11:38, 02:13, 10:36, 11:08]

מערך של עצמים - לולאות

לולאת foreach

```
// הפעולה מקבלת מערך שעונים  
// ומספר דקות שיש להפחית  
// הפעולה מפחיתה את הדקות מכל אחד מהשעונים  
static void Subtract (Clock[] arr, int minutes)  
{  
    foreach (Clock c in arr)  
    {  
        c.Subtract(minutesToSubtract);  
    }  
}
```

פעולה שמשנה איברי מערך אינה צריכה להחזיר ערך. ההפניה למערך שמורה במקום הזימון, השינוי נעשה בערכי המערך עצמו, בזיכרון ה heap

```
Clock[] arr = BuildRandomArray(7);  
Print(arr);  
Subtract(arr, 120);
```

[10:07, 00:45, 19:47, 11:38, 02:13, 10:36, 11:08]

מערך של עצמים - לולאות

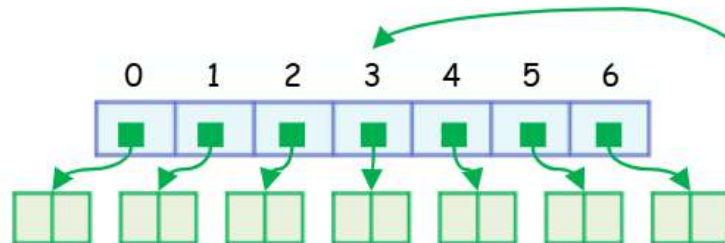
לולאת foreach

```
// הפעולה מקבלת מערך שעונים  
// ומספר דקות שיש להפחית  
// הפעולה מפחיתה את הדקות מכל אחד מהשעונים  
static void Subtract (Clock[] arr, int minutes)  
{  
    foreach (Clock c in arr)  
    {  
        c.Subtract(minutesToSubtract);  
    }  
}
```

פעולה שמשנה איברי מערך אינה צריכה להחזיר ערך. ההפניה למערך שמורה במקום הזימון, השינוי נעשה בערכי המערך עצמו, בזיכרון ה heap

```
Clock[] arr = BuildRandomArray(7);  
Print(arr);  
Subtract(arr, 120);
```

[10:07, 00:45, 19:47, 11:38, 02:13, 10:36, 11:08]



מערך של עצמים - לולאות

לולאת foreach

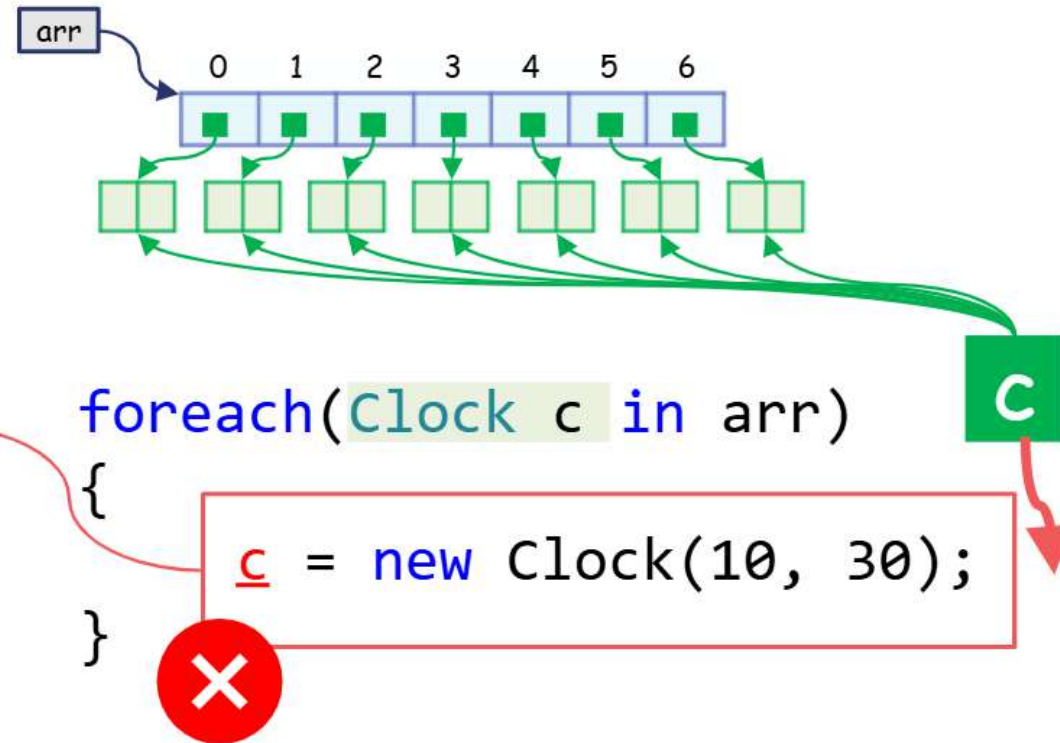
```
//פעולה מקבלת מערך שעונים  
//ומספר דקות שיש להפחית  
//הפעולה מפחיתה את הדקות מכל אחד מהשעונים  
static void Subtract (Clock[] arr, int minutes)  
{  
    foreach(Clock c in arr)  
    {  
        c.Subtract(minutesToSubtract);  
    }  
}
```

פעולה שמשנה איברי מערך אינה צריכה להחזיר ערך. ההפניה למערך שמורה במקום הזימון, השינוי נעשה בערכי המערך עצמו, בזיכרון הheap

```
Clock[] arr = BuildRandomArray(7);  
Print(arr);  
Subtract(arr, 120);  
Print(arr);
```

```
[10:07, 00:45, 19:47, 11:38, 02:13, 10:36, 11:08]  
[08:07, 22:45, 17:47, 09:38, 00:13, 08:36, 09:08]
```


מערך של עצמים – לולאות – foreach לולאת



זהירות!
לא ניתן לבצע השמה
למערך בלולאת
foreach
שגיאת קומפילציה!



לולאת foreach

- משמשת, בין השאר, למעבר על איברי מערך – מהאיבר הראשון (באינדקס 0) ועד האחרון
- המנגנון – יצירת הפניה נוספת המקבלת כל פעם את ערכו של התא התורן
- מספקת דרך נוחה וקריאה לגישה לכל איבר, ומייתרת את הצורך להשתמש באינדקסים של המערך
- לא ניתן לבצע השמה לתא במערך, כיוון שההשמה תתבצע למשתנה הלולאה ולא לתא במערך
- לא ניתן לבצע מעבר חלקי או בסדר הפוך על תאי המערך

לולאת foreach

- משמשת, בין השאר, למעבר על איברי מערך – מהאיבר הראשון (באינדקס 0) ועד האחרון
- המנגנון – יצירת הפניה נוספת המקבלת כל פעם את ערכו של התא התורן
- מספקת דרך נוחה וקריאה לגישה לכל איבר, ומייתרת את הצורך להשתמש באינדקסים של המערך
- לא ניתן לבצע השמה לתא במערך, כיוון שההשמה תתבצע למשתנה הלולאה ולא לתא במערך
- לא ניתן לבצע מעבר חלקי או בסדר הפוך על תאי המערך
- לא ניתן להתייחס לתא אחר במערך (כמו למשל לצורך בדיקת סדר מיון או סימטריה)

תכונות מחלקה

○ מפעל השעונים, מבקש לתת לכל שעון מספר מזהה ייחודי:

```
class Clock
{
    private int hours, minutes;
    private int idNum;

    public Clock(int hours, int minutes, int idNum )
    {
        SetHours(hours);
        SetMinutes(minutes);
        this.idNum = idNum;
    }
    public Clock() : this(0, 0, ? )
    {
    }
    public Clock(Clock other)
    {
        this.hours = other.hours;
        this.minutes = other.minutes;
        this.idNum = ? ;
    }
    ...
}
```

איך נבטיח שהמספר יהיה ייחודי?

איזה ערך נשלח כאן?

תכונות מחלקה

```
class Clock
{
    private int hours, minutes;
    private int idNum;

    public Clock(int hours, int minutes, int idNum)
    {
        SetHours(hours);
        SetMinutes(minutes);
        this.idNum = idNum;
    }
    public Clock() : this(0, 0, ?)
    {
    }
    public Clock(Clock other)
    {
        this.hours = other.hours;
        this.minutes = other.minutes;
        this.idNum = ? ;
    }
    ...
}
```

איך נבטיח שהמספר
יהיה ייחודי?

איזה ערך נשלח כאן?

איזה ערך נקבע כאן?

○ מפעל השעונים, מבקש לתת לכל שעון
מספר מזהה ייחודי:

תכונות מחלקה

```
class Clock
{
    private int hours, minutes;
    private int idNum;

    public Clock(int hours, int minutes, int idNum )
    {
        SetHours(hours);
        SetMinutes(minutes);
        this.idNum = idNum;
    }
    public Clock() : this(0, 0, ? )
    {
    }
    public Clock(Clock other)
    {
        this.hours = other.hours;
        this.minutes = other.minutes;
        this.idNum = ? ;
    }
    ...
}
```

איך נבטיח שהמספר
יהיה ייחודי?

איזה ערך נשלח כאן?

איזה ערך נקבע כאן?

○ מפעל השעונים, מבקש לתת לכל שעון מספר מזהה ייחודי:

○ המחלקה חייבת לבצע בקרה על המספרים, כך שהם יהיו ייחודים

○ לו היה לנו מונה אחד לכל העצמים, היינו יכולים בכל בניית עצם, לתת לו את הערך הנוכחי במונה, ולקדם את המונה

○ כך נבטיח שהמספרים המזהים יהיו ייחודיים!

תזכורת - תכונות מופע (Instance Attribute)

- עד כה הכרנו תכונות מופע
- תכונות אלו משוכפלות עבור כל עצם מהמחלקה, ולכל עצם יש את אותם משתנים
- תכונות מופע מקבלות ערך ראשוני בפעולה הבונה
- מתוך המחלקה ניתן לגשת לתכונות אלו ע"י `this`, המכיל את ההפניה לעצם הנוכחי, ואפילו ללא `this`, כאשר אין חשש לאי בהירות או התנגשות עם משתנה אחר בשם זהה
- תכונות מופע מוגדרות במחלקה, ובד"כ נגדיר אותן כ `private`, ונספק להן פעולות קובעות ומאחזרות ע"פ הצורך

```
class Clock
{
    private int hours, minutes;

    private int idNum;
```

תכונה סטטית

- תכונה סטטית היא משתנה אחד משותף עבור כל המחלקה
- לכל עצם יש גישה למשתנה זה
- אתחול – בשורת ההצהרה

```
class Clock
{
    private int hours, minutes;
    private static int count = 0;
    private int idNum;

    public Clock(int hours, int minutes)
    {
        SetHours(hours);
        SetMinutes(minutes);
        count++;
        this.idNum = count;
    }
}
```

תכונה סטטית

- תכונה סטטית היא משתנה אחד משותף עבור כל המחלקה
- לכל עצם יש גישה למשתנה זה
- אתחול – בשורת ההצהרה

```
class Clock
```

```
{  
    private int hours, minutes;  
    private static int count = 0;  
    private int idNum;  
  
    public Clock(int hours, int minutes)  
    {  
        SetHours(hours);  
        SetMinutes(minutes);  
        count++;  
        this.idNum = count;  
    }  
}
```

ביצירת עצם, נעדכן את
המשתנה הסטטי, וזה יהיה
המספר הייחודי של העצם
החדש

תכונה סטטית

- תכונה סטטית היא משתנה אחד משותף עבור כל המחלקה
- לכל עצם יש גישה למשתנה זה
- אתחול – בשורת ההצהרה
- לא ניתן לפנות למשתנה סטטי ע"י this, והפנייה היא ע"י שמו בלבד

```
class Clock
```

```
{  
    private int hours, minutes;  
    private static int count = 0;  
    private int idNum;  
  
    public Clock(int hours, int minutes)  
    {  
        SetHours(hours);  
        SetMinutes(minutes);  
        count++;  
        this.idNum = count;  
    }  
}
```

ביצירת עצם, נעדכן את
המשתנה הסטטי, וזה יהיה
המספר הייחודי של העצם
החדש

```
public Clock() : this(0, 0) {}
```

הבנאי הראשון יקצה
את המספר הייחודי

תכונה סטטית

- תכונה סטטית היא משתנה אחד משותף עבור כל המחלקה
- לכל עצם יש גישה למשתנה זה
- אתחול – בשורת ההצהרה
- לא ניתן לפנות למשתנה סטטי ע"י this, והפנייה היא ע"י שמו בלבד

תכונה סטטית

```
class Clock
{
    private int hours, minutes;
    private static int count = 0;
    private int idNum;

    public Clock(int hours, int minutes)
    {
        SetHours(hours);
        SetMinutes(minutes);
        count++;
        this.idNum = count;
    }

    public Clock() : this(0, 0) {}

    public Clock(Clock other)
    {
        this.hours = other.hours;
        this.minutes = other.minutes;
        count++;
        this.idNum = count;
    }
    ...
}
```

ביצירת עצם, נעדכן את
המשתנה הסטטי, וזה יהיה
המספר הייחודי של העצם
החדש

הבנאי הראשון יקצה
את המספר הייחודי

- תכונה סטטית היא משתנה אחד משותף עבור כל המחלקה
- לכל עצם יש גישה למשתנה זה
- אתחול – בשורת ההצהרה
- לא ניתן לפנות למשתנה סטטי ע"י this, והפנייה היא ע"י שמו בלבד

```
class Clock
```

```
{
```

```
    private int hours, minutes;
```

```
    private static int count = 0;
```

```
    private int idNum;
```

```
    public Clock(int hours, int minutes)
```

```
    {
```

```
        SetHours(hours);
```

```
        SetMinutes(minutes);
```

```
        count++;
```

```
        this.idNum = count;
```

```
    }
```

```
    public Clock() : this(0, 0) {}
```

```
    public Clock(Clock other)
```

```
    {
```

```
        this.hours = other.hours;
```

```
        this.minutes = other.minutes;
```

```
        count++;
```

```
        this.idNum = count;
```

```
    }
```

```
    ...
```

```
}
```

ביצירת עצם, נעדכן את
המשתנה הסטטי, וזה יהיה
המספר הייחודי של העצם
החדש

הבנאי הראשון יקצה
את המספר הייחודי

תכונה סטטית

- תכונה סטטית היא משתנה אחד משותף עבור כל המחלקה
- לכל עצם יש גישה למשתנה זה
- אתחול – בשורת ההצהרה
- לא ניתן לפנות למשתנה סטטי ע"י this, והפנייה היא ע"י שמו בלבד
- נוסף למחלקה פעולות אחזור וקביעה לתכונה idNum, ונתקן את פעולת ToString

תכונה סטטית

```
class Clock
{
    private int hours, minutes;
    private static int count = 0;
    private int idNum;

    public Clock(int hours, int minutes)
    {
        SetHours(hours);
        SetMinutes(minutes);
        count++;
        this.idNum = count;
    }

    public Clock() : this(0, 0) {}
    public Clock(Clock other)
    {
        this.hours = other.hours;
        this.minutes = other.minutes;
        count++;
        this.idNum = count;
    }
    ...
}
```

ID: 1 14:30

```
Clock c1 = new Clock(14, 30);
Console.WriteLine(c1);
Clock c2 = new Clock(c1);
```

idNum	1
hours	14
minutes	30

idNum	2
hours	14
minutes	30

Clock.count	2
-------------	---

תכונה סטטית

```
class Clock
{
    private int hours, minutes;
    private static int count = 0;
    private int idNum;

    public Clock(int hours, int minutes)
    {
        SetHours(hours);
        SetMinutes(minutes);
        count++;
        this.idNum = count;
    }

    public Clock() : this(0, 0) {}

    public Clock(Clock other)
    {
        this.hours = other.hours;
        this.minutes = other.minutes;
        count++;
        this.idNum = count;
    }
    ...
}
```

ID: 1 14:30
ID: 2 14:30

```
Clock c1 = new Clock(14, 30);
Console.WriteLine(c1);
Clock c2 = new Clock(c1);
Console.WriteLine(c2);
Clock c3 = new Clock();
```

idNum	1
hours	14
minutes	30

idNum	2
hours	14
minutes	30

idNum	0
hours	0
minutes	0

Clock.count	2
-------------	---


```
class Clock
```

```
{
```

```
    private int hours, minutes;  
    private static int count = 0;  
    private int idNum;
```

```
    public Clock(int hours, int minutes)
```

```
    {
```

```
        SetHours(hours);  
        SetMinutes(minutes);  
        count++;  
        this.idNum = count;
```

```
    }
```

```
    public Clock() : this(0, 0) {}
```

```
    public Clock(Clock other)
```

```
    {
```

```
        this.hours = other.hours;  
        this.minutes = other.minutes;  
        count++;  
        this.idNum = count;
```

```
    }
```

```
    ...
```

```
}
```

תכונה סטטית

```
ID: 1 14:30  
ID: 2 14:30  
ID: 3 00:00
```

```
Clock c1 = new Clock(14, 30);  
Console.WriteLine(c1);  
Clock c2 = new Clock(c1);  
Console.WriteLine(c2);  
Clock c3 = new Clock();  
Console.WriteLine(c3);
```

idNum	1
hours	14
minutes	30

idNum	2
hours	14
minutes	30

idNum	3
hours	0
minutes	0

Clock.count	3
-------------	---

תכונה סטטית

```
Clock c1 = new Clock(14, 30);
Console.WriteLine(c1);
Clock c2 = new Clock(c1);
Console.WriteLine(c2);
Clock c3 = new Clock();
Console.WriteLine(c3);

for (int h = 12, m = 15 ; h < 18; h++, m+=5)
{
    Clock c = new Clock(h, m);
    Console.WriteLine(c);
}
```

```
ID: 1 14:30
ID: 2 14:30
ID: 3 00:00
ID: 4 12:15
ID: 5 13:20
ID: 6 14:25
ID: 7 15:30
ID: 8 16:35
ID: 9 17:40
```

```

class Clock
{
    private int hours, minutes;

    private static int count = 0;

    private int idNum;

    public Clock(int hours, int minutes) {...}

    public Clock() : this(0, 0) {}

    public Clock(Clock other) {...}

    public static int GetCount()
    {
        return count;
    }

    ...
}

```

תכונה סטטית - אחזור

- פעולת אחזור לתכונה סטטית תהיה פעולה סטטית
- אין אפשרות להתייחס בה לעצם כלשהו ע"י this
- זימון פעולת אחזור סטטית, כמו זימון כל פעולה סטטית ממחלקה אחרת – ע"י שם המחלקה נקודה פעולה: Clock.GetCount()
- ניתן לכתוב פעולות סטטיות נוספות במחלקה, וגם הן אינן פעולות של העצם

```

Console.WriteLine("Count = " + Clock.GetCount());
Clock c1 = new Clock(14, 30);
Console.WriteLine("Count = " + Clock.GetCount());
Clock c2 = new Clock(c1);
Console.WriteLine("Count = " + Clock.GetCount());

```

```

Count = 0
Count = 1
Count = 2

```

תכונות מחלקה (תכונות סטטיות)

- לתכונה סטטית יהיה עותק אחד בלבד עבור כל העצמים שיווצרו מהמחלקה
- תכונה סטטית שייכת למחלקה, ולכן כל העצמים מאותה מחלקה יכולים לקרוא ולשנות תכונה זו
- הגישה לתכונה סטטית תעשה ישירות וללא שימוש ב- `this`
- תכונה סטטית נוצרת ומקבלת ערך לפני יצירת העצמים מהמחלקה (לפני יצירת העצם הראשון)
- ניתן לשנות את ערכה תוך שימוש בפעולות מופע או פעולות סטטיות

פעולות סטטיות במחלקה

- פעולה סטטית היא פעולה הנכתבת בתוך מחלקה, אך אין צורך לייצר עצם על מנת להפעיל אותה
- פעולה סטטית קשורה לוגית למחלקה, אך אינה מתבססת על נתוניו של עצם מסוים
- קריאה לפעולה סטטית מתבצעת באמצעות ציון שם המחלקה
- פעולה סטטית יכולה לגשת אך ורק לתכונות הסטטיות של המחלקה

מחלקות שירות – מחלקות סטטיות

- ישנן מחלקות שרק נותנות שירותים, כלומר יש להן אוסף פעולות ללא תכונות וללא פעולה בונה
- במחלקה כזו כל הפעולות הן סטטיות, שכן אם אין תכונות, אין משמעות לעצם
- יתכן ומחלקה זו גם תכיל אוסף של קבועים סטטיים
- מחלקת Math ומחלקת Console הן דוגמה למחלקות כאלו

```
static void Main(string[] args)
{
    Console.WriteLine(Math.Min(6, 3));
    Console.WriteLine(Math.PI);
    Console.WriteLine(Math.Ceiling(3.2));
    Console.WriteLine(Math.Pow(2, 4));
}
```


מחלקות שירות – מחלקות סטטיות

- ישנן מחלקות שרק נותנות שירותים, כלומר יש להן אוסף פעולות ללא תכונות וללא פעולה בונה
- במחלקה כזו כל הפעולות הן סטטיות, שכן אם אין תכונות, אין משמעות לעצם
- יתכן ומחלקה זו גם תכיל אוסף של קבועים סטטיים
- מחלקת Math ומחלקת Console הן דוגמה למחלקות כאלו

```
static void Main(string[] args)
{
    Console.WriteLine(Math.Min(6, 3));
    Console.WriteLine(Math.PI);
    Console.WriteLine(Math.Ceiling(3.2));
    Console.WriteLine(Math.Pow(2, 4));
}
```

```
3
3.14159265358979
4
16
```

```

class Pupil
{
    private string name;
    private int id;
    private static int numOfPupil;

    public Pupil(string name)
    { ... }
    public Pupil(Pupil other)
    { ... }

    public string GetName()
    { ... }
    public void SetName(string name)
    { ... }
    public string GetID()
    { ... }

    public static int GetNumOfPupil()
    { ... }

    public override string ToString()
    { ... }
}

```

איך לכתוב מחלקה - סיכום

- הגדרות תכונות כפרטיות - `private`.
- תכונות יכולות להיות תכונות של מופע או של מחלקה.
- כתיבת פעולת אחזור (`Get`) לכל תכונה שנרצה לחשוף.
- כתיבת פעולה קובעת (`Set`) לכל תכונה שנרצה לאפשר את עדכונה. מימוש פעולת `Set` יכולול בדיקת תקינות (אם קיימת).
- כתיבת פעולות בונות:
 - יקבלו כפרמטרים את ערכי התכונות, איתן נרצה לאתחל את העצם.
 - ניתן ורצוי להעמיס פעולות בונות.
 - ניתן להשתמש בפעולות ה-`Set` הקיימות.
 - פעולה בונה מעתיקה תקבל עצם מסוג המחלקה.
- מימוש פעולת `ToString`
- פעולות נוספות לפי צרכי המחלקה.